



Курс-квест Middle Node.js Developer+

Модуль 1 - Врата Node.js (основы платформы)

Урок 1 - Что такое Node.js, где он применяется.

Шаг 01:

Миссия 1

Ты пришёл на собеседование, и рекрутер спрашивает: "Что же такое Node.js?"

Подсказка: Вспомни: Node.js — это не язык, а среда.

Ответ:

- Среда выполнения JavaScript вне браузера
- Новый язык программирования
- Библиотека для Python
- Фреймворк для Java

Шаг 02:

Миссия 2

Программист Антон запустил сервер на Node.js и удивился, что он обслуживает сотни запросов одновременно. Что делает это возможным?

Подсказка: Думай про Event Loop.

Ответ:

- Неблокирующая, событийно-ориентированная модель
- Node.js создает поток на каждый запрос

- Node.js запускает код параллельно на всех ядрах автоматически
- Node.js компилируется в машинный код как C++

🔗 Шаг 03:

Миссия 3

Мария разрабатывает Telegram-бота на Node.js. Почему именно Node.js часто выбирают для ботов?

Подсказка: Вспомни про работу с сетью и асинхронность.

Ответ:

- **Удобна асинхронная работа с API и сетью**
- Node.js содержит встроенный Telegram SDK
- Node.js работает только с мессенджерами
- Node.js автоматически переводит текст сообщений

🔗 Шаг 04:

Миссия 4

Компания решила написать интернет-магазин. Почему Node.js может быть хорошим выбором для backend?

Подсказка: Подумай про скорость и масштабируемость.

Ответ:

- **Высокая производительность при обработке большого числа одновременных соединений**
- Node.js автоматически генерирует дизайн сайта
- Node.js работает только с фронтендом
- Node.js не требует знаний JavaScript

🔗 Шаг 05:

Миссия 5

Разработчик Лена хочет понять, чем Node.js отличается от браузера. Что верно?

Подсказка: Думай про доступ к файловой системе.

Ответ:

- **Node.js может работать с файловой системой напрямую**
- Node.js умеет только рисовать интерфейс
- Node.js встроен в каждый браузер
- Node.js не поддерживает модули

🔗 Шаг 06:

Миссия 6

На митапе спросили: "Почему Node.js любят фронтенд-разработчики?"

Подсказка: Подумай про единый язык на фронте и бэке.

Ответ:

- **Потому что можно писать и фронтенд, и бэкенд на JavaScript**
- Потому что Node.js автоматически пишет CSS
- Потому что Node.js встроен в HTML
- Потому что Node.js работает только в браузере

🔧 Шаг 07:

Миссия 7

📖 Твоя команда обсуждает Node.js. Сопоставь понятия с их описаниями.

Подсказка: Вспомни, чем Node.js отличается от языка и фреймворка.

Ответ:

- **Node.js** — Среда выполнения JavaScript вне браузера
- **JavaScript** — Язык программирования
- **Express** — Фреймворк для создания веб-приложений на Node.js
- **NPM** — Менеджер пакетов для Node.js

🔧 Шаг 08:

Миссия 8

📖 Разработчик Саша хочет разобраться, какие задачи чаще всего решают на Node.js. Сопоставь направление и пример использования.

Подсказка: Подумай: real-time, API, автоматизация.

Ответ:

- **Real-time приложения** — Чаты и онлайн-игры
- **API сервер** — Backend для мобильного приложения
- **Автоматизация** — Скрипт для конвертации файлов
- **Streaming** — Передача видео в реальном времени

🔧 Шаг 09:

Миссия 9

📖 Тебе дали задание подготовить проект. Сопоставь встроенный модуль Node.js и его назначение.

Подсказка: Подумай про fs, http, path.

Ответ:

- **fs** — Работа с файловой системой
- **http** — Создание веб-сервера
- **path** — Работа с путями к файлам
- **os** — Информация о системе

🔧 Шаг 10:

Миссия 10

📖 Твой друг путается в терминах. Сопоставь технологию с её ролью.

Подсказка: Запомни: Node.js — это не база данных и не язык.

Ответ:

- **Node.js** — Среда выполнения JS
- **MongoDB** — База данных NoSQL
- **PostgreSQL** — Реляционная база данных
- **React** — Фреймворк для фронтенда

🔧 Шаг 11:

Миссия 11

📖 На лекции прозвучали ключевые термины. Сопоставь их.

Подсказка: Event Loop управляет асинхронностью.

Ответ:

- **Event Loop** — Механизм обработки событий и колбэков
- **Асинхронность** — Выполнение задач без блокировки основного потока
- **Callback** — Функция, вызываемая по завершению другой операции
- **Promise** — Объект для работы с результатом асинхронной операции

🔗 Шаг 12:

Миссия 12

Расположи шаги запуска простого Node.js проекта в правильном порядке.

Подсказка: Сначала нужна папка, потом package.json, потом код.

Ответ:

- Создать папку проекта
- Инициализировать npm (package.json)
- Написать index.js
- Запустить node index.js

🔗 Шаг 13:

Миссия 13

Ты решил написать сервер. Расположи шаги по созданию HTTP-сервера на Node.js в правильном порядке.

Подсказка: Вспомни: require → createServer → listen.

Ответ:

- Подключить модуль http
- Создать сервер с http.createServer
- Написать обработчик запросов
- Запустить сервер методом listen

🔗 Шаг 14:

Миссия 14

Команда хочет развернуть Node.js проект. Расположи шаги деплоя на сервере.

Подсказка: Сначала клонируют проект, потом ставят зависимости.

Ответ:

- Клонировать репозиторий
- Установить зависимости npm install
- Настроить окружение (.env)
- Запустить приложение через node или pm2

🔗 Шаг 15:

Миссия 15

Ты решил создать CLI на Node.js. Расположи шаги разработки в правильном порядке.

Подсказка: Сначала package.json, потом shebang и publish.

Ответ:

- Создать package.json с bin
- Добавить shebang в js-файл
- Реализовать функционал
- Опубликовать пакет в npm

 Урок 2 - Event Loop и модель ввода-вывода.

 Шаг 01:

Миссия 1

 Программист Аня пишет чат-бота на Node.js. Она заметила, что один запрос к базе данных не блокирует другие операции. Как это возможно?

Подсказка: Вспомните, что Node.js не ждет завершения операции, а продолжает выполнение кода.

Ответ:

- **Благодаря неблокирующему вводу-выводу и Event Loop**
- Так как Node.js автоматически клонирует процессы
- Из-за того, что каждая операция запускается в отдельном потоке
- Потому что JavaScript компилируется в машинный код

 Шаг 02:

Миссия 2

 Разработчик Игорь удивился: его функция с setTimeout отработала позже, чем ожидалось. Почему?

Подсказка: Таймеры не всегда выполняются строго по времени, на них влияет состояние очереди задач.

Ответ:

- **Event Loop был занят выполнением других задач**
- setTimeout работает только в браузере, а не в Node.js
- Функция случайно удалась из памяти
- Node.js автоматически увеличивает задержку для оптимизации

 Шаг 03:

Миссия 3

 Катя делает сервер. Она заметила, что тяжёлый цикл for блокирует остальные запросы. Почему?

Подсказка: JavaScript в Node.js выполняется в одном основном потоке.

Ответ:

- **Потому что Node.js работает на одном потоке**
- Потому что цикл всегда запускается в отдельном процессе
- Из-за того, что база данных блокирует цикл
- Потому что Event Loop не обрабатывает коллбэки

 Шаг 04:

Миссия 4

🧑💻 Разработчик Миша хочет понять разницу: что именно попадает в Event Loop в первую очередь?

Подсказка: Сравните macro-task и micro-task.

Ответ:

- **Micro-task (например, Promise) выполняются раньше, чем setTimeout**
- setTimeout всегда быстрее Promise
- Event Loop выполняет только таймеры
- Очередь micro-task запускается только по кнопке

💡 Шаг 05:

Миссия 5

🧑💻 Настя изучает асинхронность. Она спросила: почему колбэки файловой системы не выполняются мгновенно?

Подсказка: Эти операции выполняются в пуле потоков (libuv) и возвращаются в Event Loop.

Ответ:

- **Потому что операции выполняются в пуле потоков и возвращаются позже**
- Файловая система всегда работает синхронно
- Node.js не поддерживает работу с файлами
- Колбэки блокируются JavaScript-движком V8

💡 Шаг 06:

Миссия 6

🧑💻 Денис изучает очередь событий. Сопоставьте термины с их описанием.

Подсказка: Подумайте: Promise и таймеры относятся к разным типам задач.

Ответ:

- **Micro-task** — Выполняются сразу после текущего стека, до перехода к таймерам.
- **Macro-task** — Крупные задачи, такие как setTimeout или setInterval.
- **libuv** — Библиотека, управляющая асинхронными операциями ввода-вывода.
- **Event Loop** — Механизм, который проверяет очередь и выполняет задачи по порядку.

💡 Шаг 07:

Миссия 7

🧑💻 Полина хочет закрепить теорию. Сопоставьте функцию с её местом в Event Loop.

Подсказка: Вспомните различие между Promise и setTimeout.

Ответ:

- **setTimeout** — Попадает в очередь macro-task.
- **Promise.then** — Обрабатывается в очереди micro-task.
- **fs.readFile** — Выполняется в пуле потоков libuv, а затем возвращается в Event Loop.
- **process.nextTick** — Выполняется даже раньше micro-task.

💡 Шаг 08:

Миссия 8

🧑💻 Тимур проверяет знания коллег. Сопоставьте понятие с аналогией.

Подсказка: Подумайте о повседневных примерах очередей.

Ответ:

- **Event Loop** — Очередь на кассе супермаркета, где кассир обслуживает по порядку.
- **Micro-task** — Как приоритетная очередь для VIP-покупателей.
- **Macro-task** — Обычные клиенты, которые ждут своей очереди.
- **libuv** — Команда работников склада, выполняющих заказы в фоновом режиме.

🔮 Шаг 09:

Миссия 9

🧑💻 Марина решает тест: сопоставьте этап Event Loop с его назначением.

Подсказка: В Node.js есть фазы: timers, poll, check и др.

Ответ:

- **Timers** — Обрабатывает setTimeout и setInterval.
- **Pending callbacks** — Выполняет колбэки от операций ввода-вывода.
- **Poll** — Ожидание новых событий и их обработка.
- **Check** — Выполнение setImmediate.

🔮 Шаг 10:

Миссия 10

🧑💻 Артём объясняет другу: почему Promise и process.nextTick ведут себя по-разному?

Сопоставьте механизм с его особенностью.

Подсказка: Один из них имеет более высокий приоритет.

Ответ:

- **process.nextTick** — Выполняется сразу после текущей операции, раньше любых micro-task.
- **Promise.then** — Выполняется в очереди micro-task.
- **setImmediate** — Запускается в фазе check после poll.
- **setTimeout** — Запускается в фазе timers.

🔮 Шаг 11:

Миссия 11

🧑💻 Оля готовит интервью. Сопоставьте задачу с её типом.

Подсказка: Задачи делятся на синхронные, micro-task и macro-task.

Ответ:

- **console.log** — Синхронная задача.
- **Promise.resolve().then(...)** — Micro-task.
- **setTimeout(..., 0)** — Macro-task.
- **process.nextTick(...)** — Особая очередь, выполняется до micro-task.

🔮 Шаг 12:

Миссия 12

🧑💻 Никита запускает код. Расположите события в правильном порядке выполнения:

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
```

Подсказка: Сначала выполняется синхронный код, потом micro-task, а затем таймеры.

Ответ:

- A
- D
- C
- B

💡 Шаг 13:

Миссия 13

🧑💻 Кира учится оптимизировать код. Расставьте шаги выполнения Event Loop по порядку.

Подсказка: Вспомните стандартные фазы: timers → pending → poll → check.

Ответ:

- Timers
- Pending callbacks
- Poll
- Check

💡 Шаг 14:

Миссия 14

🧑💻 Антон тестирует код. Расположите вывод в правильном порядке:

```
setImmediate(() => console.log('X'));
setTimeout(() => console.log('Y'), 0);
console.log('Z');
```

Подсказка: Сначала синхронный код, потом таймеры и setImmediate.

Ответ:

- Z
- Y
- X

💡 Шаг 15:

Миссия 15

🧑💻 София тренируется. Расположите этапы работы с асинхронным кодом:

Ответ:

- Выполнение синхронного кода
- Обработка micro-task
- Обработка macro-task
- Вывод результата

Подсказка: Сначала стек, потом micro-task, потом очередь таймеров.

- Выполнение синхронного кода
- Обработка micro-task
- Обработка macro-task
- Вывод результата

🔧 Шаг 16:

Миссия 16

👤 Алексей проверяет код с promise и таймерами. Расставьте порядок:

```
setTimeout(() => console.log('1'), 0);  
Promise.resolve().then(() => console.log('2'));  
console.log('3');
```

Подсказка: Сначала синхронный код, потом promise, затем таймер.

Ответ:

- 3
- 2
- 1

📖 Урок 3 - Установка, npm, nrm.

🔧 Шаг 01:

Миссия 1

Молодой разработчик Иван установил Node.js, но не понимает, зачем ему менеджер версий npm. Для чего чаще всего используют npm?

Подсказка: Иногда проекты требуют разные версии Node.js, и без переключения они конфликтуют.

Ответ:

- Для установки и переключения между разными версиями Node.js
- Для запуска тестов внутри проекта
- Для ускорения npm-скриптов
- Для проверки синтаксиса JavaScript

🔧 Шаг 02:

Миссия 2

Алина пытается запустить старый проект, где требовалась Node.js 14. У неё стоит Node.js 20. Как ей правильно действовать?

Подсказка: В таких случаях npm приходит на помощь.

Ответ:

- Установить Node.js 14 через npm и переключиться на неё
- Удалить Node.js 20 и вручную скачать Node.js 14
- Запустить проект напрямую, версии всегда совместимы
- Использовать npm для смены версии Node.js

🔧 Шаг 03:

Миссия 3

Петя заметил, что при запуске команды npm install появляется папка node_modules. Для чего она нужна?

Подсказка: Вспомни, где хранятся все установленные зависимости проекта.

Ответ:

- Хранит все установленные пакеты и их зависимости

- Содержит только системные бинарные файлы
- Это временная папка, которая очищается при выходе
- В ней хранятся только тестовые файлы проекта

🔧 Шаг 04:

Миссия 4

Катя случайно удалила папку `node_modules` . Как восстановить зависимости?

Подсказка: Все необходимые версии пакетов хранятся в специальном файле.

Ответ:

- **Выполнить `npm install`**
- Скачать все пакеты вручную из GitHub
- Перезапустить компьютер
- Создать папку `node_modules` заново вручную

🔧 Шаг 05:

Миссия 5

Денис запускает проект и видит сообщение: `npm ERR! missing script: start` . Что это значит?

Подсказка: Проверь, есть ли в `package.json` секция `scripts`.

Ответ:

- **В `package.json` нет скрипта `start`**
- Не установлен Node.js
- Неверно указан порт сервера
- Ошибка в коде `index.js`

🔧 Шаг 06:

Миссия 6

Оля решила попробовать Yarn вместо npm. Какой командой она установит зависимости из `package.json`?

Подсказка: В Yarn используется своя короткая команда.

Ответ:

- **`yarn install`**
- `yarn get`
- `npm install`
- `yarn add-all`

🔧 Шаг 07:

Миссия 7

Максим добавил новый пакет в проект с помощью `npm install axios` . Какой файл обновился?

Подсказка: Этот файл описывает зависимости проекта.

Ответ:

- **`package.json`**
- `server.js`
- `index.html`
- `README.md`

🔧 Шаг 08:

Миссия 8

Ирина хочет установить пакет только для разработки (например, nodemon). Какой флаг она должна использовать?

Подсказка: Вспомни разницу между зависимостями для продакшн и dev.

Ответ:

- **--save-dev**
- --only-dev
- --test
- --local

🔧 Шаг 09:

Миссия 9

Сергей заметил, что в проекте появился файл package-lock.json . Для чего он нужен?

Подсказка: Этот файл фиксирует конкретные версии пакетов, чтобы проект у всех работал одинаково.

Ответ:

- **Фиксирует версии всех установленных пакетов**
- Хранит список авторов проекта
- Определяет структуру каталогов
- Заменяет package.json

🔧 Шаг 10:

Миссия 10

Сопоставьте команды npm с их назначением:

Подсказка: Каждая команда выполняет уникальную задачу.

Ответ:

- **npm init** — Создать новый package.json
- **npm install** — Установить зависимости из package.json
- **npm uninstall** — Удалить пакет
- **npm update** — Обновить пакеты

🔧 Шаг 11:

Миссия 11

Сопоставьте менеджеры пакетов с их особенностями:

Подсказка: npm, yarn и pnpm работают по-разному.

Ответ:

- **npm** — Стандартный менеджер, устанавливается с Node.js
- **yarn** — Быстрее устанавливает пакеты, имеет lock-файл yarn.lock
- **pnpm** — Экономит место благодаря ссылкам на общую кешированную папку

🔧 Шаг 12:

Миссия 12

Сопоставьте команды yarn с их аналогами в npm:

Подсказка: yarn часто короче.

Ответ:

- **yarn install** — npm install
- **yarn add package** — npm install package
- **yarn remove package** — npm uninstall package
- **yarn run script** — npm run script

🔮 Шаг 13:

Миссия 13

Сопоставьте файлы проекта с их назначением:

Подсказка: Каждый файл играет уникальную роль.

Ответ:

- **package.json** — Список зависимостей и скриптов проекта
- **package-lock.json** — Фиксированные версии пакетов
- **node_modules** — Файлы всех установленных библиотек
- **README.md** — Описание проекта

🔮 Шаг 14:

Миссия 14

Расставьте шаги установки Node.js через nvm в правильном порядке:

Подсказка: Сначала установка nvm, затем установка и выбор версии Node.js.

Ответ:

- Установить nvm
- Установить нужную версию Node.js
- Переключиться на эту версию
- Проверить версию командой node -v

🔮 Шаг 15:

Миссия 15

Определи правильный порядок запуска нового проекта:

Подсказка: package.json создаётся в начале.

Ответ:

- Создать папку проекта
- Перейти в папку проекта
- Выполнить npm init
- Установить нужные пакеты

🔮 Шаг 16:

Миссия 16

Расставь шаги при добавлении нового пакета:

Подсказка: После установки обновляются файлы.

Ответ:

- Выполнить npm install package
- Обновляется package.json

- Обновляется package-lock.json
- Появляется папка в node_modules

Урок 4 - CommonJS vs ES Modules.

Шаг 01:

Миссия 1

Молодой разработчик Артём подключает библиотеку в Node.js. Он использует синтаксис require() . К какому формату модулей он обратился?

Подсказка: Вспомни: require() появился в Node.js первым и до сих пор работает.

Ответ:

- **CommonJS**
- ES Modules
- AMD
- UMD

Шаг 02:

Миссия 2

Настя пишет современный код и использует import ... from . Какой это стандарт модулей?

Подсказка: Этот стандарт пришёл из браузеров и стал нативным в Node.js позже.

Ответ:

- CommonJS
- **ES Modules**
- SystemJS
- AMD

Шаг 03:

Миссия 3

Дима пытается использовать require() в файле с расширением .mjs , но получает ошибку. Почему?

Подсказка: Node.js строго разделяет форматы файлов по расширению.

Ответ:

- **Файл .mjs поддерживает только ES Modules**
- Файл .mjs поддерживает только CommonJS
- В Node.js нельзя использовать require
- Ошибка в синтаксисе import

Шаг 04:

Миссия 4

Марина открыла проект и увидела type:"module" в package.json. Что это значит?

Подсказка: Этот флаг влияет на то, как Node.js интерпретирует .js файлы.

Ответ:

- **Все .js файлы будут интерпретироваться как ES Modules**
- Файлы автоматически компилируются в CommonJS
- Node.js переключается в режим strict

- Можно использовать только .mjs файлы

🔗 Шаг 05:

Миссия 5

Егор подключает модуль через `import`, но Node.js не находит файл без расширения. Почему?

Подсказка: CommonJS умеет автоматически подставлять расширения, а вот ES Modules более строгие.

Ответ:

- **В ES Modules нужно явно указывать расширение**
- Node.js не поддерживает `import`
- Файл не находится в `node_modules`
- Файл нельзя импортировать из подкаталога

🔗 Шаг 06:

Миссия 6

Алексей хочет использовать переменную `__dirname` в ES Modules, но получает `undefined`. Что нужно сделать?

Подсказка: В ES Modules нет этих переменных напрямую, их можно сгенерировать вручную.

Ответ:

- **Создать `__dirname` через `import.meta.url` и `path`**
- Использовать `require('path').dirname`
- Включить флаг `commonjs` в `package.json`
- Подключить библиотеку `babel`

🔗 Шаг 07:

Миссия 7

Компания разрабатывает библиотеку. Какой формат модулей лучше выбрать, если нужен максимальный уровень совместимости с существующими проектами Node.js?

Подсказка: Старые проекты ещё не везде перешли на ESM.

Ответ:

- **CommonJS**
- ES Modules
- UMD
- AMD

🔗 Шаг 08:

Миссия 8

Сопоставьте ключевые особенности CommonJS и ES Modules.

Подсказка: Подумайте, какой синтаксис у каждого формата и как они выполняются.

Ответ:

- **CommonJS** — Использует `require()` и `module.exports`
- **ES Modules** — Использует `import/export`
- **CommonJS загрузка** — Выполняется синхронно
- **ES Modules загрузка** — Выполняется асинхронно

Шаг 09:

Миссия 9

Сопоставьте расширения файлов с правильным типом модулей.

Подсказка: Одно расширение работает только для ESM, другое — по умолчанию CommonJS.

Ответ:

- **.cjs** — Файл CommonJS
- **.mjs** — Файл ES Modules
- **.js без type:module** — CommonJS
- **.js с type:module** — ES Modules

Шаг 10:

Миссия 10

Расставь шаги подключения модуля в CommonJS в правильном порядке.

Подсказка: Сначала пишется модуль, потом экспорт, потом require.

Ответ:

- Написать функцию в модуле
- Экспортировать функцию через module.exports
- Импортировать модуль с помощью require
- Вызвать функцию из подключенного модуля

Шаг 11:

Миссия 11

Расставь шаги подключения модуля в ES Modules в правильном порядке.

Подсказка: Подумай: сначала код, потом export, потом import.

Ответ:

- Создать функцию в модуле
- Экспортировать её с помощью export
- Импортировать её с помощью import
- Вызвать функцию

Шаг 12:

Миссия 12

Расставь шаги настройки проекта для использования ES Modules.

Подсказка: В package.json есть важный параметр.

Ответ:

- Открыть package.json
- Добавить "type":"module"
- Создать файл с расширением .js
- Использовать синтаксис import/export

Шаг 13:

Миссия 13

Расставь шаги, чтобы симитировать __dirname в ES Modules.

Подсказка: Вспомни про `import.meta.url` и модуль `path`.

Ответ:

- Импортировать модуль `path`
- Получить путь через `import.meta.url`
- Преобразовать его с помощью `fileURLToPath`
- Взять `dirname` через `path.dirname`

Урок 5 - Работа с `package.json`.

Шаг 01:

Миссия 1

Молодой разработчик Саша создал новый проект. Какой файл появляется автоматически после команды `npm init -y` ?

Подсказка: Это основной файл конфигурации Node.js проекта.

Ответ:

- **`package.json`**
- `index.js`
- `node_modules.json`
- `config.js`

Шаг 02:

Миссия 2

Аня заметила поле `name` в `package.json`. Для чего оно используется?

Подсказка: Подумай о публикации пакета в `npm`.

Ответ:

- **Для имени пакета при публикации и идентификации**
- Для указания главного файла проекта
- Для версии Node.js
- Для списка зависимостей

Шаг 03:

Миссия 3

Денис увидел поле `version` в `package.json`. Какой формат версий чаще всего используется?

Подсказка: Вспомни про семантическое версионирование: `major.minor.patch`.

Ответ:

- **SemVer (например, 1.2.3)**
- Год.месяц (например, 2023.09)
- Просто число (например, 5)
- Буквенный код

Шаг 04:

Миссия 4

Игорь хочет указать главный файл проекта. Какое поле отвечает за это в `package.json`?

Подсказка: Обычно это `index.js`, но можно задать свой путь.

Ответ:

- **main**
- scripts
- dependencies
- type

 **Шаг 05:**

Миссия 5

Маша хочет добавить скрипт запуска сервера. В какое поле package.json она должна его записать?

Подсказка: В этом разделе можно хранить свои команды npm run ...

Ответ:

- **scripts**
- devDependencies
- config
- engines

 **Шаг 06:**

Миссия 6

Виктор устанавливает библиотеку для разработки и использует --save-dev . В какое поле package.json попадёт зависимость?

Подсказка: Такие зависимости нужны только во время разработки, а не в продакшене.

Ответ:

- **devDependencies**
- dependencies
- peerDependencies
- optionalDependencies

 **Шаг 07:**

Миссия 7

Компания разрабатывает библиотеку и хочет указать, с какими версиями Node.js она совместима. Какое поле package.json использовать?

Подсказка: Это поле часто содержит ограничения вида ">=14".

Ответ:

- **engines**
- dependencies
- main
- os

 **Шаг 08:**

Миссия 8

Сопоставьте поля package.json с их назначением.

Подсказка: Эти поля встречаются в каждом проекте.

Ответ:

- **name** — Имя пакета
- **version** — Версия пакета
- **description** — Краткое описание проекта
- **author** — Автор проекта

🔮 **Шаг 09:**

Миссия 9

Сопоставьте поля package.json и их содержимое.

Подсказка: Вспомни, какие поля отвечают за зависимости.

Ответ:

- **dependencies** — Зависимости для продакшена
- **devDependencies** — Зависимости для разработки
- **peerDependencies** — Зависимости, ожидаемые у конечного пользователя
- **optionalDependencies** — Необязательные зависимости

🔮 **Шаг 10:**

Миссия 10

Сопоставьте команды npm и их действие в контексте package.json.

Подсказка: Обрати внимание, какие команды изменяют файл.

Ответ:

- **npm install express** — Добавляет зависимость в dependencies
- **npm install jest --save-dev** — Добавляет зависимость в devDependencies
- **npm init -y** — Создает package.json по умолчанию
- **npm run build** — Запускает скрипт build из package.json

🔮 **Шаг 11:**

Миссия 11

Сопоставьте поля package.json и их использование при публикации пакета.

Подсказка: npm использует эти данные при загрузке в реестр.

Ответ:

- **keywords** — Ключевые слова для поиска
- **license** — Тип лицензии проекта
- **repository** — Ссылка на репозиторий
- **bugs** — Ссылка для сообщений об ошибках

🔮 **Шаг 12:**

Миссия 12

Сопоставьте поля package.json с примерами значений.

Подсказка: Пример: { "type": "module" }.

Ответ:

- **type** — module
- **main** — index.js
- **scripts** — { "start": "node app.js" }

- **engines** — `>=14`

Шаг 13:

Миссия 13

Расставь шаги создания `package.json` вручную.

Подсказка: Сначала создаётся файл, потом вносятся данные.

Ответ:

- Создать файл `package.json`
- Добавить поле `name`
- Добавить поле `version`
- Добавить поле `main`

Шаг 14:

Миссия 14

Расставь шаги добавления зависимости через `npm`.

Подсказка: `npm` сначала загружает пакет, потом обновляет `package.json` и устанавливает файлы.

Ответ:

- Ввести команду `npm install express`
- `npm` обновляет `package.json`
- `npm` обновляет `package-lock.json`
- `npm` устанавливает файлы в `node_modules`

Шаг 15:

Миссия 15

Расставь шаги выполнения скрипта `npm run dev`.

Подсказка: Сначала ищется скрипт в `package.json`.

Ответ:

- `npm` ищет `scripts.dev` в `package.json`
- Запускается команда, указанная в `scripts.dev`
- Выводится результат работы команды
- Процесс завершается

Шаг 16:

Миссия 16

Расставь шаги публикации пакета в `npm`.

Подсказка: Подумай: сначала создаётся учётная запись, потом пакет готовится и публикуется.

Ответ:

- Создать учётную запись в `npm`
- Настроить `package.json`
- Выполнить команду `npm login`
- Выполнить команду `npm publish`

Урок 6 - Мини-босс CLI «Hello Adventurer» + пакет в npm (локально).

Шаг 01:

Миссия 1

Разработчик Костя хочет создать простую CLI-программу. Какой shebang нужно поставить в начале файла?

Подсказка: Он подсказывает системе, какой интерпретатор использовать.

Ответ:

- **#!/usr/bin/env node**
- `#!/usr/bin/env bash`
- `#!/usr/bin/env js`
- `#!/bin/sh node`

Шаг 02:

Миссия 2

Лена пишет CLI и хочет выводить приветствие. Какой метод `console` подойдёт?

Подсказка: Он выводит информацию в `stdout`.

Ответ:

- **`console.log`**
- `console.error`
- `console.trace`
- `console.debug`

Шаг 03:

Миссия 3

Вадим хочет сделать так, чтобы его CLI можно было запускать как команду. В какое поле `package.json` нужно указать путь к файлу?

Подсказка: Это поле хранит привязку команд к файлам.

Ответ:

- **`bin`**
- `scripts`
- `main`
- `cli`

Шаг 04:

Миссия 4

Сергей протестировал свой CLI локально. Как установить пакет в систему для теста?

Подсказка: Эта команда создаёт глобальную ссылку на пакет.

Ответ:

- **`npm link`**
- `npm install -g .`
- `npm init -y`
- `npm run cli`

🔧 Шаг 05:

Миссия 5

Миша случайно забыл поставить права на выполнение файла CLI. Какая команда исправит ситуацию в Linux?

Подсказка: Она делает файл исполняемым.

Ответ:

- **chmod +x filename**
- chown +x filename
- run filename
- exec filename

🔧 Шаг 06:

Миссия 6

Катя создала CLI. Какая команда позволит проверить, что команда hello доступна глобально?

Подсказка: После npm link просто вызови её в терминале.

Ответ:

- **hello**
- node hello.js
- npm run hello
- ./hello.js

🔧 Шаг 07:

Миссия 7

Илья хочет указать в package.json несколько CLI-команд. Как это сделать?

Подсказка: Можно указать объект, где ключ — команда, значение — путь.

Ответ:

- **bin как объект с несколькими парами ключ-значение**
- main со списком файлов
- scripts с массивом
- dependencies с alias

🔧 Шаг 08:

Миссия 8

Сопоставьте поле package.json с его назначением при создании CLI.

Подсказка: Подумай о связке команды и файла.

Ответ:

- **bin** — Указывает команду и путь к файлу
- **main** — Определяет точку входа пакета
- **name** — Имя пакета
- **version** — Версия пакета

🔧 Шаг 09:

Миссия 9

Сопоставьте команды npm с их эффектом при работе над CLI.

Подсказка: Вспомни, какая команда создаёт символическую ссылку.

Ответ:

- **npm link** — Создаёт глобальную ссылку на пакет
- **npm install -g .** — Устанавливает пакет глобально
- **npm init -y** — Создаёт package.json
- **npm run start** — Запускает скрипт start

🧭 Шаг 10:

Миссия 10

Сопоставьте элементы CLI и их примеры.

Подсказка: Вспомни shebang и вызов команды.

Ответ:

- **Shebang** — `#!/usr/bin/env node`
- **Команда** — `hello`
- **Вывод** — `Hello Adventurer`
- **Файл** — `cli.js`

🧭 Шаг 11:

Миссия 11

Сопоставьте проблему и способ её решения в CLI.

Подсказка: Каждая ошибка решается определённым действием.

Ответ:

- **Команда not found** — Проверить поле `bin` и `npm link`
- **Нет прав на запуск файла** — Использовать `chmod +x`
- **Пустой вывод** — Добавить `console.log`
- **Неверный shebang** — Использовать `#!/usr/bin/env node`

🧭 Шаг 12:

Миссия 12

Сопоставьте npm поле и пример значения.

Подсказка: Эти значения часто встречаются в CLI-проектах.

Ответ:

- **bin** — `{"hello":"cli.js"}`
- **scripts** — `{"start":"node index.js"}`
- **main** — `index.js`
- **author** — `Ivan Petrov`

🧭 Шаг 13:

Миссия 13

Расставь шаги создания простого CLI-файла.

Подсказка: Сначала shebang, затем код, потом права.

Ответ:

- Добавить shebang `#!/usr/bin/env node`
- Написать `console.log("Hello Adventurer")`

- Сохранить файл cli.js
- Выдать права chmod +x cli.js

💡 Шаг 14:

Миссия 14

Расставь шаги добавления CLI в package.json.

Подсказка: Подумай: сначала создать package.json, потом добавить bin.

Ответ:

- Создать package.json
- Добавить поле bin с командой
- npm link
- Вызвать команду hello

💡 Шаг 15:

Миссия 15

Расставь шаги подготовки пакета к публикации.

Подсказка: Учти, что нужно заполнить ключевые поля package.json.

Ответ:

- Указать name, version, description
- Добавить поле bin
- Проверить работу CLI через npm link
- Сделать git init и commit

💡 Шаг 16:

Миссия 16

Расставь шаги теста CLI локально.

Подсказка: Последний шаг — запустить команду.

Ответ:

- Написать cli.js
- Добавить bin в package.json
- Выполнить npm link
- Запустить hello

📁 Модуль 2 - 🌲 Асинхронный лес

📖 Урок 1 - Колбэки и их недостатки.

💡 Шаг 01:

Миссия 1

Разработчик Петя пишет асинхронный код в Node.js. Какой аргумент у callback-функции обычно передаётся первым?

Подсказка: Это соглашение помогает сразу обрабатывать ошибки.

Ответ:

- Ошибка (error)
- Результат

- null
- Promise

🔮 Шаг 02:

Миссия 2

Маша использует функцию `fs.readFile`. Какой аргумент она должна передать последним?

Подсказка: В асинхронных функциях последний параметр — это особая функция.

Ответ:

- **Callback-функцию**
- Путь к файлу
- Кодировку
- Опции в виде объекта

🔮 Шаг 03:

Миссия 3

Игорь написал несколько вложенных callback'ов для последовательных операций. Как называют такую проблему?

Подсказка: Визуально это напоминает ёлочку из отступов.

Ответ:

- **Callback hell**
- Promise chaining
- Event loop
- Deadlock

🔮 Шаг 04:

Миссия 4

Оля пытается отладить callback-ад, но теряется в коде. Почему вложенные колбэки плохо масштабируются?

Подсказка: Подумай о читаемости и управлении ошибками.

Ответ:

- **Код становится трудным для чтения и сопровождения**
- Код выполняется быстрее
- Callback'и блокируют поток
- Вложенность ускоряет отладку

🔮 Шаг 05:

Миссия 5

Дима получил ошибку в асинхронном коде, но она оказалась глубоко внутри вложенных колбэков. В чём сложность?

Подсказка: Ошибки нужно обрабатывать на каждом уровне.

Ответ:

- **Ошибки сложно пробросить наружу**
- Ошибки автоматически обрабатываются
- Ошибки блокируют event loop
- Ошибки исчезают без следа

🔧 Шаг 06:

Миссия 6

Андрей задумался: можно ли заменить callback'и на что-то более удобное? Какой стандарт появился после них?

Подсказка: Этот подход работает с then/catch.

Ответ:

- **Promise**
- EventEmitter
- Thread
- Cluster

🔧 Шаг 07:

Миссия 7

Наташа объясняет новичкам минус callback'ов. Какое утверждение верное?

Подсказка: Вспомни про архитектуру и отладку.

Ответ:

- **Callback'и усложняют поддержку больших проектов**
- Callback'и делают код синхронным
- Callback'и быстрее промисов
- Callback'и автоматически обрабатывают ошибки

🔧 Шаг 08:

Миссия 8

Сопоставьте функцию и её типичное применение с callback'ом.

Подсказка: Это стандартные асинхронные операции Node.js.

Ответ:

- **fs.readFile** — Чтение файла
- **setTimeout** — Отложенный вызов
- **http.get** — Получение данных по сети
- **fs.writeFile** — Запись в файл

🔧 Шаг 09:

Миссия 9

Вы пишете CLI для управления задачами и столкнулись с колбэками. Расположите шаги обработки файла с задачами в правильном порядке:

1. Прочитать файл tasks.json асинхронно
2. Обработать полученные данные в колбэке
3. Преобразовать JSON в массив объектов
4. Вывести задачи в консоль

Подсказка: Данные сначала нужно получить, затем распарсить, а потом вывести.

Ответ:

- Прочитать файл tasks.json асинхронно
- Обработать полученные данные в колбэке
- Преобразовать JSON в массив объектов

- Вывести задачи в консоль

Шаг 10:

Миссия 10

Ваня оптимизирует код и решает заменить вложенные колбэки на более чистую структуру. Расположите шаги перехода:

1. Выделение повторяющихся частей в отдельные функции
2. Передача функций как аргументов вместо анонимных колбэков
3. Сокращение вложенности
4. Повышение читаемости и тестируемости

Подсказка: Подумайте, как постепенно упрощается структура кода.

Ответ:

- Выделение повторяющихся частей в отдельные функции
- Передача функций как аргументов вместо анонимных колбэков
- Сокращение вложенности
- Повышение читаемости и тестируемости

Шаг 11:

Миссия 11

Настя пишет код для загрузки файлов. Расположите шаги, чтобы понять, где именно возникает ошибка в цепочке колбэков:

1. Добавить `console.log` в начало первого колбэка
2. Добавить отладку внутри второго колбэка
3. Проверить корректность данных перед третьим вызовом
4. Определить источник ошибки

Подсказка: Отладка должна быть пошаговой: сверху вниз.

Ответ:

- Добавить `console.log` в начало первого колбэка
- Добавить отладку внутри второго колбэка
- Проверить корректность данных перед третьим вызовом
- Определить источник ошибки

Шаг 12:

Миссия 12

Вы переводите колбэки на промисы. Расположите шаги в правильной последовательности:

1. Оборачиваем асинхронную функцию в `Promise`
2. Заменяем колбэки на `resolve/reject`
3. Используем `then/catch` для обработки результата
4. Получаем более линейный и читаемый код

Подсказка: Сначала создаётся промис, затем подключается цепочка обработки.

Ответ:

- Оборачиваем асинхронную функцию в `Promise`
- Заменяем колбэки на `resolve/reject`

- Используем then/catch для обработки результата
- Получаем более линейный и читаемый код

🔔 Шаг 13:

Миссия 13

Алексей решает провести ревизию старого проекта. Расположите шаги, чтобы заменить устаревшие колбэки на современный подход:

1. Определить места с глубокой вложенностью колбэков
2. Переписать их через промисы
3. Заменить промисы на async/await
4. Добиться максимально читаемого и поддерживаемого кода

Подсказка: Подумайте о поэтапной эволюции: от колбэков к промисам и далее к async/await.

Ответ:

- Определить места с глубокой вложенностью колбэков
- Переписать их через промисы
- Заменить промисы на async/await
- Добиться максимально читаемого и поддерживаемого кода

📖 Урок 2 - Promise и цепочки.

🔔 Шаг 01:

Миссия 1

Вы написали функцию, которая возвращает Promise, выполняющийся через 2 секунды. Какой вариант корректно создает Promise?

Подсказка: Подумайте о синтаксисе Promise и обязательных аргументах.

Ответ:

- **new Promise((resolve, reject) => setTimeout(() => resolve('done'), 2000))**
- Promise.delay(2000, 'done')
- setTimeout(Promise.resolve('done'), 2000)
- new Promise(setTimeout(() => 'done', 2000))

🔔 Шаг 02:

Миссия 2

Вы вызываете несколько асинхронных функций, которые возвращают Promise. Как лучше всего дождаться их выполнения одновременно?

Подсказка: Существует метод, который принимает массив промисов и выполняется, когда все они завершатся.

Ответ:

- **Promise.all([promise1, promise2, promise3])**
- promise1.then().then().then()
- setTimeout(() => ..., 1000)
- Promise.race([promise1, promise2, promise3])

🔔 Шаг 03:

Миссия 3

Функция `fetchData` возвращает `Promise`. Как обработать успешный результат и ошибку?

Подсказка: Обработка ошибок идет через `.catch` после `.then`.

Ответ:

- `fetchData().then(result => console.log(result)).catch(err => console.error(err))`
- `fetchData().catch(result => console.log(result))`
- `fetchData().then(result => console.log(result))`
- `fetchData(err => console.error(err))`

Шаг 04:

Миссия 4

 Сопоставьте методы `Promise` с их поведением

Методы:

Ответ:

- `Promise.all`
- `Promise.race`
- `Promise.resolve`
- `Promise.reject`

Описание:

1. Создает промис, который сразу успешен с указанным значением
2. Возвращает промис, который завершится первой завершившейся из группы
3. Возвращает промис, который завершается ошибкой с указанной причиной
4. Возвращает промис, который успешно завершится, когда все переданные промисы завершатся

Подсказка: Обратите внимание на отличие между `collective` и `first-resolved` промисами.

- **`Promise.all`** — Возвращает промис, который успешно завершится, когда все переданные промисы завершатся
- **`Promise.race`** — Возвращает промис, который завершится первой завершившейся из группы
- **`Promise.resolve`** — Создает промис, который сразу успешен с указанным значением
- **`Promise.reject`** — Возвращает промис, который завершается ошибкой с указанной причиной

Шаг 05:

Миссия 5

 Сопоставьте ключевые свойства `Promise`

Ответ:

- `Pending`
- `Fulfilled`
- `Rejected`
- `Thenable`

1. Состояние до завершения операции
2. Состояние успешного выполнения
3. Состояние завершения с ошибкой
4. Объект с методом `.then`

Подсказка: Сначала промис ждет, потом может выполняться или провалиться.

- **Pending** — Состояние до завершения операции
- **Fulfilled** — Состояние успешного выполнения
- **Rejected** — Состояние завершения с ошибкой
- **Thenable** — Объект с методом `.then`

🔮 Шаг 06:

Миссия 6

📄 Сопоставьте события в цепочке промисов с их действием

Ответ:

- `.then()`
 - `.catch()`
 - `.finally()`
 - `chained promise`
1. Выполняется после любого результата, успешного или ошибки
 2. Создает новую цепочку промисов
 3. Обрабатывает успешный результат
 4. Обрабатывает ошибку

Подсказка: Цепочка строится через `then`, ошибки ловятся через `catch`, `finally` всегда срабатывает.

- **`.then()`** — Обрабатывает успешный результат
- **`.catch()`** — Обрабатывает ошибку
- **`.finally()`** — Выполняется после любого результата, успешного или ошибки
- **`chained promise`** — Создает новую цепочку промисов

🔮 Шаг 07:

Миссия 7

У вас есть функция, возвращающая промис, и вы хотите выполнить ее трижды последовательно. Что выбрать?

Подсказка: Цепочка промисов выполняется в порядке `.then()`.

Ответ:

- **`doSomething().then(doSomething).then(doSomething)`**
- `Promise.all([doSomething(), doSomething(), doSomething()])`
- `doSomething(); doSomething(); doSomething();`
- `setTimeout(doSomething, 0)`

🔮 Шаг 08:

Миссия 8

Вы хотите обрабатывать ошибки в цепочке из нескольких промисов. Что лучше использовать?

Подсказка: Ошибка может произойти на любом шаге цепочки.

Ответ:

- **Добавить `.catch()` в конец цепочки**
- Использовать `try {} catch {}` вокруг каждого `then`
- Игнорировать ошибки
- Вызвать `Promise.resolve()` для каждой функции

Шаг 09:

Миссия 9

 Сопоставьте ситуации с подходящими методами Promise

Ответ:

- Дождаться всех данных одновременно
- Получить результат первого ответа
- Создать сразу успешный промис
- Создать сразу отклоненный промис

1. Promise.all
2. Promise.race
3. Promise.resolve
4. Promise.reject

Подсказка: Все промисы одновременно или первый завершившийся — разные методы.

- **Дождаться всех данных одновременно** — Promise.all
- **Получить результат первого ответа** — Promise.race
- **Создать сразу успешный промис** — Promise.resolve
- **Создать сразу отклоненный промис** — Promise.reject

Шаг 10:

Миссия 10

 Сопоставьте термины цепочки промисов с их описанием

Ответ:

- Промис
 - Цепочка
 - Ошибка
 - Асинхронная функция
1. Объект, который представляет результат асинхронной операции
 2. Последовательность промисов, где результат одного передается следующему
 3. Неожиданное завершение с reject
 4. Функция, которая всегда возвращает промис

Подсказка: Промис — результат операции, цепочка — порядок вызова, ошибки нужно обрабатывать.

- **Промис** — Объект, который представляет результат асинхронной операции
- **Цепочка** — Последовательность промисов, где результат одного передается следующему
- **Ошибка** — Неожиданное завершение с reject
- **Асинхронная функция** — Функция, которая всегда возвращает промис

Шаг 11:

Миссия 11

Функция apiCall возвращает промис. Вы хотите повторить вызов при ошибке. Что использовать?

Подсказка: Обработчик catch может возвращать новый промис для повторного вызова.

Ответ:

- `apiCall().catch(() => apiCall())`

- `try { apiCall(); } catch { apiCall(); }`
- `Promise.resolve(apiCall())`
- `setTimeout(apiCall, 1000)`

🔗 Шаг 12:

Миссия 12

Вы хотите выполнить несколько промисов последовательно и сохранить результаты в массив. Как лучше?

Подсказка: Используйте цепочку `then` и аккумулируйте результаты.

Ответ:

- `promises.reduce((acc, fn) => acc.then(res => fn().then(r => [...res, r])), Promise.resolve([]))`
- `Promise.all(promises)`
- `promises.forEach(fn => fn())`
- `setTimeout(() => promises.map(fn => fn()), 0)`

🔗 Шаг 13:

Миссия 13

Расположите шаги обработки цепочки промисов в правильном порядке:

1. Создание начального промиса
2. Добавление обработчиков `.then` и `.catch`
3. Возврат нового значения или промиса внутри `.then`
4. Обработка ошибки в `.catch`
5. Использование `.finally` для завершения

Подсказка: Сначала создается промис, потом добавляем обработчики, затем возвращаем результаты, ловим ошибки, и в конце выполняем финальные действия.

Ответ:

- Создание начального промиса
- Добавление обработчиков `.then` и `.catch`
- Возврат нового значения или промиса внутри `.then`
- Обработка ошибки в `.catch`
- Использование `.finally` для завершения

🔗 Шаг 14:

Миссия 14

Расположите последовательность обработки нескольких асинхронных запросов:

1. Отправка первого запроса
2. Обработка результата первого запроса
3. Отправка второго запроса после первого
4. Обработка второго результата
5. Обработка ошибок

Подсказка: Сначала отправляем запрос, обрабатываем результат, затем следующий, и не забываем о `catch`.

Ответ:

- Отправка первого запроса
- Обработка результата первого запроса
- Отправка второго запроса после первого
- Обработка второго результата
- Обработка ошибок

Урок 3 - asyncawait.

Шаг 01:

Миссия 1

Разработчик пишет функцию `fetchUserData`, которая возвращает промис. Как правильно вызвать её с помощью `async/await`?

Подсказка: `await` можно использовать только внутри `async` функции.

Ответ:

- **`async function getData() { const user = await fetchUserData(); }`**
- `const user = await fetchUserData();`
- `fetchUserData().then(user => return user);`
- `const user = fetchUserData();`

Шаг 02:

Миссия 2

Вы хотите обработать ошибку внутри `async` функции при вызове промиса `fetchOrders`. Как правильно это сделать?

Подсказка: Для обработки ошибок используйте `try/catch`.

Ответ:

- **`async function loadOrders() { try { const orders = await fetchOrders(); } catch(err) { console.error(err); } }`**
- `fetchOrders().catch(err => console.error(err));`
- `try { await fetchOrders(); }`
- `async function loadOrders() { const orders = fetchOrders(); }`

Шаг 03:

Миссия 3

Разработчик хочет выполнить несколько асинхронных операций параллельно с `async/await`. Как это сделать?

Подсказка: Используйте `Promise.all` с `await` для параллельного выполнения.

Ответ:

- **`const [user, orders] = await Promise.all([fetchUser(), fetchOrders()]);`**
- `const user = await fetchUser(); const orders = await fetchOrders();`
- `await fetchUser(); await fetchOrders();`
- `Promise.all([fetchUser(), fetchOrders()]).then();`

Шаг 04:

Миссия 4

 Сопоставьте конструкции async/await с их назначением

Ответ:

- async function
- await
- try/catch
- Promise.all

1. Объявляет функцию как асинхронную, возвращает промис
2. Ожидает результат промиса перед продолжением выполнения
3. Обрабатывает ошибки при выполнении async кода
4. Позволяет ждать несколько промисов одновременно

Подсказка: Обратите внимание на различие между синтаксисом функции, ожиданием промиса и обработкой ошибок.

- **async function** — Объявляет функцию как асинхронную, возвращает промис
- **await** — Ожидает результат промиса перед продолжением выполнения
- **try/catch** — Обрабатывает ошибки при выполнении async кода
- **Promise.all** — Позволяет ждать несколько промисов одновременно

Шаг 05:

Миссия 5

 Сопоставьте типичные ошибки при использовании async/await с их последствиями

Ответ:

- await вне async функции
- необработанный reject
- последовательный вызов промисов вместо параллельного
- забыли return внутри async функции

1. Синтаксическая ошибка, код не выполнится
2. Ошибка не будет поймана, приложение может упасть
3. Увеличенное время выполнения
4. Функция возвращает undefined вместо значения

Подсказка: Подумайте о правилах async/await и результатах выполнения промисов.

- **await вне async функции** — Синтаксическая ошибка, код не выполнится
- **необработанный reject** — Ошибка не будет поймана, приложение может упасть
- **последовательный вызов промисов вместо параллельного** — Увеличенное время выполнения
- **забыли return внутри async функции** — Функция возвращает undefined вместо значения

Шаг 06:

Миссия 6

 Сопоставьте жизненные ситуации разработчиков с использованием async/await

Ответ:

- Обновление данных профиля
- Загрузка нескольких ресурсов одновременно
- Повторный запрос при ошибке сети

- Логирование результата после завершения операции

1. await внутри try/catch для обработки ошибок
2. Promise.all с await
3. try/catch с повторным вызовом функции
4. finally блок

Подсказка: Подумайте, где нужна обработка ошибок, а где параллельное выполнение.

- **Обновление данных профиля** — await внутри try/catch для обработки ошибок
- **Загрузка нескольких ресурсов одновременно** — Promise.all с await
- **Повторный запрос при ошибке сети** — try/catch с повторным вызовом функции
- **Логирование результата после завершения операции** — finally блок

Шаг 07:

Миссия 7

Разработчик хочет дождаться завершения нескольких асинхронных функций по очереди и сохранить их результаты. Как лучше сделать?

Подсказка: Для последовательного выполнения используйте обычный await внутри цикла.

Ответ:

- **for (const fn of funcs) { results.push(await fn()); }**
- await funcs.forEach(fn => fn());
- Promise.all(funcs.map(fn => fn()));
- funcs.map(fn => await fn());

Шаг 08:

Миссия 8

Вы используете async/await и хотите логировать результат выполнения, независимо от успеха или ошибки. Как сделать?

Подсказка: finally блок выполнится всегда.

Ответ:

- **try { await doTask(); } catch(e) {} finally { console.log('done'); }**
- await doTask().then().catch();
- console.log(await doTask());
- doTask().finally();

Шаг 09:

Миссия 9

 Сопоставьте методы обработки нескольких промисов с их результатом при async/await

Ответ:

- Promise.all
 - Promise.allSettled
 - Promise.race
 - Promise.any
1. Ждет выполнения всех промисов, завершение с ошибкой отклоняет все
 2. Ждет выполнения всех промисов и возвращает результаты, успешные и ошибки
 3. Возвращает результат первого завершившегося промиса

4. Возвращает результат первого успешно завершившегося промиса

Подсказка: Обратите внимание на различия между `all`, `allSettled`, `race` и `any`.

- **Promise.all** — Ждет выполнения всех промисов, завершение с ошибкой отклоняет все
- **Promise.allSettled** — Ждет выполнения всех промисов и возвращает результаты, успешные и ошибки
- **Promise.race** — Возвращает результат первого завершившегося промиса
- **Promise.any** — Возвращает результат первого успешно завершившегося промиса

🔧 Шаг 10:

Миссия 10

📄 Сопоставьте этапы работы с `async/await` с правильной последовательностью

Ответ:

- Создание `async` функции
 - Вызов промиса с `await`
 - Обработка ошибок
 - Логирование результата
1. Создание `async` функции
 2. Вызов промиса с `await`
 3. Обработка ошибок через `try/catch`
 4. Логирование результата через `finally` или после `await`

Подсказка: Сначала создаем функцию, потом ждём результат, затем ловим ошибки и логируем.

- **Создание `async` функции** — Создание `async` функции
- **Вызов промиса с `await`** — Вызов промиса с `await`
- **Обработка ошибок** — Обработка ошибок через `try/catch`
- **Логирование результата** — Логирование результата через `finally` или после `await`

🔧 Шаг 11:

Миссия 11

Вы хотите, чтобы `async` функция возвращала значение после нескольких `await`. Что нужно не забыть?

Подсказка: Результат `async` функции всегда оборачивается в промис.

Ответ:

- **Использовать `return` перед значением**
- Просто вызвать значение без `return`
- Вызвать `console.log` вместо `return`
- Возвратить `await` без значения

🔧 Шаг 12:

Миссия 12

Разработчик хочет повторять `async` вызов до успешного результата. Что выбрать?

Подсказка: Цикл с `try/catch` и `await` внутри подойдет для повторения.

Ответ:

- **`while(!success) { try { await doTask(); success = true; } catch(e) {} }`**
- `doTask().then();`

- `setTimeout(doTask, 1000);`
- `await doTask();`

🔗 Шаг 13:

Миссия 13

Расположите шаги последовательного выполнения async операций:

1. Объявление async функции
2. Вызов первой async операции с `await`
3. Вызов второй async операции с `await`
4. Обработка ошибок `try/catch`
5. Возврат результата

Подсказка: Сначала функция, потом шаги внутри нее, с обработкой ошибок и возвратом результата.

Ответ:

- Объявление async функции
- Вызов первой async операции с `await`
- Вызов второй async операции с `await`
- Обработка ошибок `try/catch`
- Возврат результата

🔗 Шаг 14:

Миссия 14

Расположите шаги параллельного выполнения нескольких async функций с `await`:

1. Создание массива промисов
2. Передача массива в `Promise.all` с `await`
3. Обработка ошибок `try/catch`
4. Логирование результатов

Подсказка: Сначала формируем промисы, затем ждём все с `Promise.all`, ловим ошибки и выводим результаты.

Ответ:

- Создание массива промисов
- Передача массива в `Promise.all` с `await`
- Обработка ошибок `try/catch`
- Логирование результатов

📖 Урок 4 - Error handling.

🔗 Шаг 01:

Миссия 1

Разработчик делает запрос к API и хочет поймать возможные ошибки. Как правильно обработать промис?

Подсказка: Для обработки ошибок используйте `.catch` или `try/catch` внутри async функции.

Ответ:

- `fetchData().catch(err => console.error(err));`
- `fetchData();`

- `try { fetchData(); }`
- `console.log(fetchData());`

🔗 Шаг 02:

Миссия 2

Вы используете `async/await` и хотите обработать ошибку при запросе к базе данных. Как это сделать?

Подсказка: Ошибки в `async` функциях ловятся через `try/catch`.

Ответ:

- **`async function getData() { try { const res = await dbQuery(); } catch(e) { console.error(e); } }`**
- `const res = await dbQuery();`
- `dbQuery().then();`
- `console.log(await dbQuery());`

🔗 Шаг 03:

Миссия 3

Разработчик хочет поймать ошибки сразу нескольких промисов. Что использовать?

Подсказка: Для параллельного выполнения промисов используйте `Promise.all` с обработкой ошибок.

Ответ:

- **`Promise.all([p1,p2]).catch(err => console.error(err));`**
- `await p1; await p2;`
- `p1.then().then();`
- `console.log(await p1,p2);`

🔗 Шаг 04:

Миссия 4

📄 Сопоставьте ошибки с их последствиями:

Ответ:

- Необработанный `reject`
- `await` вне `async` функции
- забыли `try/catch`
- ошибка в `Promise.all`

1. Приложение может аварийно завершиться
2. Синтаксическая ошибка, код не выполнится
3. Ошибка не будет поймана, приложение может упасть
4. Все промисы могут быть отклонены, если один промис упал

Подсказка: Подумайте, как ошибки промисов и `await` влияют на выполнение программы.

- **Необработанный `reject`** — Приложение может аварийно завершиться
- **`await` вне `async` функции** — Синтаксическая ошибка, код не выполнится
- **забыли `try/catch`** — Ошибка не будет поймана, приложение может упасть
- **ошибка в `Promise.all`** — Все промисы могут быть отклонены, если один промис упал

🔗 Шаг 05:

Миссия 5

📖 Сопоставьте методы обработки ошибок с их результатом:

Ответ:

- `.catch()`
- `try/catch`
- `finally`
- `Promise.all().catch()`

1. Ловит ошибку промиса
2. Обрабатывает ошибки внутри `asunc` функции
3. Выполняется независимо от успеха или ошибки
4. Ловит ошибку при выполнении нескольких промисов одновременно

Подсказка: Обратите внимание на различие обработки ошибок внутри `asunc` функций и у промисов.

- **`.catch()`** — Ловит ошибку промиса
- **`try/catch`** — Обрабатывает ошибки внутри `asunc` функции
- **`finally`** — Выполняется независимо от успеха или ошибки
- **`Promise.all().catch()`** — Ловит ошибку при выполнении нескольких промисов одновременно

🔗 Шаг 06:

Миссия 6

📖 Сопоставьте жизненные ситуации с типом обработки ошибок:

Ответ:

- Запрос к API с возможной ошибкой
- Обновление базы данных внутри `asunc` функции
- Выполнение нескольких промисов параллельно
- Логирование результата после операции

1. `.catch()`
2. `try/catch`
3. `Promise.all().catch()`
4. `finally` блок

Подсказка: Подумайте, где нужна обработка ошибок и где просто финальная операция.

- **Запрос к API с возможной ошибкой** — `.catch()`
- **Обновление базы данных внутри `asunc` функции** — `try/catch`
- **Выполнение нескольких промисов параллельно** — `Promise.all().catch()`
- **Логирование результата после операции** — `finally` блок

🔗 Шаг 07:

Миссия 7

Вы хотите, чтобы приложение не падало при любой ошибке промиса. Что выбрать?

Подсказка: Всегда обрабатывайте `reject` промиса через `.catch` или `try/catch`.

Ответ:

- **Добавить .catch() ко всем промисам или использовать try/catch в async функции**
- Игнорировать ошибки
- Вызывать промисы без await
- Использовать console.log без обработки ошибок

Шаг 08:

Миссия 8

Разработчик хочет повторить запрос при ошибке сети. Как правильно обработать ошибку?

Подсказка: Используйте try/catch с повторным вызовом функции.

Ответ:

- **while(!success){ try{ await fetchData(); success=true; }catch(e){} }**
- fetchData().then();
- setTimeout(fetchData,1000);
- console.log(await fetchData());

Шаг 09:

Миссия 9

 Сопоставьте ошибки Node.js с их типичными последствиями:

Ответ:

- UnhandledPromiseRejectionWarning
 - SyntaxError
 - ReferenceError
 - TypeError
1. Необработанный reject промиса
 2. Ошибка синтаксиса, код не выполнится
 3. Использование несуществующей переменной
 4. Неверный тип данных в операции

Подсказка: Подумайте, какие ошибки возникают до выполнения кода, а какие — во время выполнения.

- **UnhandledPromiseRejectionWarning** — Необработанный reject промиса
- **SyntaxError** — Ошибка синтаксиса, код не выполнится
- **ReferenceError** — Использование несуществующей переменной
- **TypeError** — Неверный тип данных в операции

Шаг 10:

Миссия 10

 Сопоставьте виды ошибок с способами их обработки:

Ответ:

- Асинхронная ошибка
- Синхронная ошибка
- Ошибка в нескольких промисах

- Логирование результата после ошибки

1. `.catch()` или `try/catch`
2. `try/catch`
3. `Promise.all().catch()`
4. `finally` блок

Подсказка: Асинхронные и синхронные ошибки требуют разного подхода.

- **Асинхронная ошибка** — `.catch()` или `try/catch`
- **Синхронная ошибка** — `try/catch`
- **Ошибка в нескольких промисах** — `Promise.all().catch()`
- **Логирование результата после ошибки** — `finally` блок

Шаг 11:

Миссия 11

Вы хотите вывести пользователю сообщение при любой ошибке промиса. Как правильно?

Подсказка: Используйте `catch` с выводом сообщения.

Ответ:

- `fetchData().catch(err => alert('Произошла ошибка'));`
- `console.log(fetchData());`
- `await fetchData();`
- `fetchData().then();`

Шаг 12:

Миссия 12

Вы хотите гарантировать выполнение кода после `async` операции независимо от ошибок. Что выбрать?

Подсказка: `finally` выполняется всегда, после `try/catch`.

Ответ:

- `try { await doTask(); } catch(e) {} finally { console.log('done'); }`
- `await doTask();`
- `doTask().then();`
- `console.log(doTask());`

Шаг 13:

Миссия 13

Расположите шаги обработки ошибки `async` функции:

1. Объявление `async` функции
2. Вызов промиса с `await`
3. Оборачивание вызова в `try/catch`
4. Логирование ошибки или результата
5. Возврат значения или `undefined`

Подсказка: Сначала функция, затем вызовы с обработкой ошибок и возврат результата.

Ответ:

- Объявление `async` функции

- Вызов промиса с `await`
- Оборачивание вызова в `try/catch`
- Логирование ошибки или результата
- Возврат значения или `undefined`

🔗 Шаг 14:

Миссия 14

Расположите шаги обработки ошибок нескольких промисов параллельно:

1. Создание массива промисов
2. Оборачивание `Promise.all` в `try/catch`
3. Вызов `Promise.all` с `await`
4. Логирование ошибок и результатов

Подсказка: Сначала формируем промисы, затем ждём их выполнения с обработкой ошибок и выводим результаты.

Ответ:

- Создание массива промисов
- Оборачивание `Promise.all` в `try/catch`
- Вызов `Promise.all` с `await`
- Логирование ошибок и результатов

📖 Урок 5 - Таймеры, микротаски, `setImmediate`.

🔗 Шаг 01:

Миссия 1

Разработчик запускает `setTimeout` и хочет понять, когда код выполнится.

Подсказка: `setTimeout` откладывает выполнение в очередь макротасок.

Ответ:

- **После завершения текущего стека вызовов и всех микротасок**
- Сразу при вызове
- До выполнения микротасок
- Никогда, если не `await`

🔗 Шаг 02:

Миссия 2

Вы используете `process.nextTick`. Когда выполняется функция?

Подсказка: `nextTick` ставит колбэк в начало очереди микротасок.

Ответ:

- **После текущего стека вызовов, до `setTimeout` и `setImmediate`**
- После `setTimeout`
- В следующем тике событийного цикла
- Сразу при объявлении

🔗 Шаг 03:

Миссия 3

Разработчик хочет выполнить код после всех I/O операций, но до следующего тика таймера. Что выбрать?

Подсказка: `setImmediate` выполняется после завершения текущего цикла событий.

Ответ:

- **`setImmediate`**
- `setTimeout(fn,0)`
- `process.nextTick`
- `await Promise.resolve()`

Шаг 04:

Миссия 4

 Сопоставьте функции с очередью выполнения:

Ответ:

- `setTimeout`
 - `process.nextTick`
 - `Promise.then`
 - `setImmediate`
1. Микротаски, выполняются сразу после текущего стека
 2. Макротаски, после микротасок и I/O
 3. Микротаски, после текущего стека, до макротасок
 4. Макротаски, после I/O операций

Подсказка: Микротаски всегда выполняются раньше макротасок.

- **`setTimeout`** — Макротаски, после микротасок и I/O
- **`process.nextTick`** — Микротаски, выполняются сразу после текущего стека
- **`Promise.then`** — Микротаски, после текущего стека, до макротасок
- **`setImmediate`** — Макротаски, после I/O операций

Шаг 05:

Миссия 5

 Сопоставьте типы колбэков с примером:

Ответ:

- Микротаска
 - Макротаска
 - I/O колбэк
 - `setImmediate` колбэк
1. `Promise.then`
 2. `setTimeout(fn,0)`
 3. `fs.readFile(..., callback)`
 4. `setImmediate(fn)`

Подсказка: Подумайте о том, что выполняется раньше: микротаски или макротаски.

- **Микротаска** — `Promise.then`
- **Макротаска** — `setTimeout(fn,0)`
- **I/O колбэк** — `fs.readFile(..., callback)`

- **setImmediate колбэк** — `setImmediate(fn)`

🔗 Шаг 06:

Миссия 6

Разработчик хочет, чтобы callback выполнялся сразу после текущего стека, но перед I/O. Что выбрать?

Подсказка: Микротаски выполняются раньше макротасок.

Ответ:

- **process.nextTick или Promise.then**
- `setTimeout(fn, 0)`
- `setImmediate(fn)`
- `fs.readFile(..., callback)`

🔗 Шаг 07:

Миссия 7

Вы хотите понять порядок выполнения в Node.js между `setTimeout`, `setImmediate` и `process.nextTick`. С чего начать?

Подсказка: Лучше всего экспериментировать с `console.log` внутри разных колбэков.

Ответ:

- **Сделать небольшую функцию с `console.log` в каждом колбэке и наблюдать порядок**
- Сразу использовать `setTimeout(0)` для всех
- Игнорировать порядок, Node сам решит
- Вызывать все через `await`

🔗 Шаг 08:

Миссия 8

📄 Сопоставьте таймеры с их свойствами:

Ответ:

- `setTimeout(fn, 0)`
- `setImmediate(fn)`
- `process.nextTick(fn)`
- `Promise.then(fn)`

1. Макротаска, выполняется после микротасок
2. Макротаска после I/O
3. Микротаска, выполняется сразу после текущего стека
4. Микротаска, выполняется после текущего стека, до макротасок

Подсказка: Разделите очередь выполнения на микротаски и макротаски.

- **`setTimeout(fn, 0)`** — Макротаска, выполняется после микротасок
- **`setImmediate(fn)`** — Макротаска после I/O
- **`process.nextTick(fn)`** — Микротаска, выполняется сразу после текущего стека
- **`Promise.then(fn)`** — Микротаска, выполняется после текущего стека, до макротасок

💡 Шаг 09:

Миссия 9

📄 Сопоставьте функции с примером использования:

Ответ:

- `process.nextTick`
 - `Promise.then`
 - `setTimeout(fn,0)`
 - `setImmediate(fn)`
1. Выполнить колбэк сразу после текущей функции
 2. Выполнить асинхронный код после завершения текущего стека
 3. Отложить выполнение на следующий цикл событий
 4. Выполнить после всех операций I/O

Подсказка: Сравните приоритет выполнения микротасок и макротасок.

- **`process.nextTick`** — Выполнить колбэк сразу после текущей функции
- **`Promise.then`** — Выполнить асинхронный код после завершения текущего стека
- **`setTimeout(fn,0)`** — Отложить выполнение на следующий цикл событий
- **`setImmediate(fn)`** — Выполнить после всех операций I/O

💡 Шаг 10:

Миссия 10

Вы хотите выполнить функцию после всех I/O операций. Что выбрать?

Подсказка: `setImmediate` гарантирует выполнение после I/O.

Ответ:

- **`setImmediate(fn)`**
- `process.nextTick(fn)`
- `Promise.then(fn)`
- `setTimeout(fn,0)`

💡 Шаг 11:

Миссия 11

Разработчик хочет гарантировать, что микротаска выполнится до макротаски. Что использовать?

Подсказка: `Promise.then` и `process.nextTick` — микротаски.

Ответ:

- **`Promise.then` или `process.nextTick`**
- `setTimeout(fn,0)`
- `setImmediate(fn)`
- `await fn`

📖 Урок 6 - Мини-босс написать асинхронный загрузчик файлов

💡 Шаг 01:

Миссия 1

Аня хочет начать писать асинхронный загрузчик файлов. Какой метод Node.js позволит ей читать файл без блокировки потока?

Подсказка: Ищите методы с колбэком или промисом.

Ответ:

- **fs.readFile с колбэком или fs.promises.readFile**
- fs.readFileSync
- console.log
- setTimeout

Шаг 02:

Миссия 2

Петя хочет обрабатывать ошибки при асинхронном чтении файла. Как правильно сделать?

Подсказка: Любой асинхронный вызов может завершиться ошибкой.

Ответ:

- **Проверять аргумент error в колбэке или использовать try/catch с async/await**
- Игнорировать ошибки
- Оборачивать код в setTimeout
- Вызывать console.log без проверки

Шаг 03:

Миссия 3

Разработчик хочет последовательно загружать несколько файлов. Какой подход лучше?

Подсказка: Последовательность важна для сохранения порядка данных.

Ответ:

- **Использовать async/await внутри цикла**
- Запускать все fs.readFile одновременно без await
- Использовать setTimeout для задержки
- Читать файлы синхронно в main thread

Шаг 04:

Миссия 4

 Сопоставьте элементы асинхронного загрузчика с их функцией

Ответ:

- fs.promises.readFile
- try/catch
- await
- console.error

1. Считывает файл асинхронно и возвращает промис
2. Обрабатывает ошибки при чтении файла
3. Ожидает завершения промиса
4. Выводит информацию об ошибке в консоль

Подсказка: Сопоставьте действия с их предназначением.

- **fs.promises.readFile** — Считывает файл асинхронно и возвращает промис
- **try/catch** — Обрабатывает ошибки при чтении файла

- **await** — Ожидает завершения промиса
- **console.error** — Выводит информацию об ошибке в консоль

Шаг 05:

Миссия 5

 Сопоставьте события загрузчика с правильным порядком действий

Ответ:

- Запуск загрузки файла
 - Ожидание завершения промиса
 - Обработка данных
 - Обработка ошибок
1. Иницируется чтение файла
 2. Если промис отклонён, выполняется блок catch
 3. Если промис выполнен успешно, данные передаются дальше
 4. Асинхронно ждём завершения операции

Подсказка: Подумайте, что происходит первым, а что последним.

- **Запуск загрузки файла** — Иницируется чтение файла
- **Ожидание завершения промиса** — Асинхронно ждём завершения операции
- **Обработка данных** — Если промис выполнен успешно, данные передаются дальше
- **Обработка ошибок** — Если промис отклонён, выполняется блок catch

Шаг 06:

Миссия 6

Игорь хочет одновременно загрузить несколько файлов и дождаться всех результатов. Как лучше сделать?

Подсказка: Есть функция, которая объединяет промисы.

Ответ:

- **Promise.all для параллельной загрузки**
- async/await по очереди без Promise.all
- setTimeout с разными интервалами
- Чтение синхронно через fs.readFileSync

Шаг 07:

Миссия 7

Марина хочет логировать прогресс загрузки файлов. Как лучше сделать?

Подсказка: Логировать после успешного завершения каждого промиса.

Ответ:

- **Выводить console.log после await для каждого файла**
- Логировать до чтения файла
- Использовать setTimeout для логирования
- Игнорировать логирование

Шаг 08:

Миссия 8

 Сопоставьте события загрузчика файлов с их назначением

Ответ:

- Запуск загрузки файла
 - Ожидание завершения промиса
 - Обработка данных
 - Обработка ошибок
1. Иницирует чтение файла
 2. Асинхронно ждёт завершения операции
 3. Данные успешно загружены и переданы дальше
 4. Промис отклонён и выполняется блок catch

Подсказка: Подумайте о последовательности событий при загрузке файла.

- **Запуск загрузки файла** — Иницирует чтение файла
- **Ожидание завершения промиса** — Асинхронно ждёт завершения операции
- **Обработка данных** — Данные успешно загружены и переданы дальше
- **Обработка ошибок** — Промис отклонён и выполняется блок catch

 **Шаг 09:**

Миссия 9

Наташа хочет, чтобы несколько файлов загружались одновременно и ждать завершения всех. Какой метод использовать?

Подсказка: Существует функция, которая объединяет несколько промисов.

Ответ:

- **Promise.all**
- async/await по очереди без Promise.all
- setTimeout с разными интервалами
- fs.readFileSync

 **Шаг 10:**

Миссия 10

Саша хочет логировать прогресс загрузки каждого файла после его завершения. Как это сделать?

Подсказка: Логировать после await или then для каждого промиса.

Ответ:

- **Выводить console.log после завершения промиса**
- Логировать до начала чтения файла
- Использовать setTimeout для логирования
- Игнорировать прогресс

 **Шаг 11:**

Миссия 11

Расположите шаги асинхронной загрузки файлов в правильном порядке:

1. Иницируем чтение файла с помощью fs.promises.readFile
2. Обрабатываем ошибки через try/catch
3. Ожидаем завершения промиса через await

4. Работаем с данными после успешного завершения

Подсказка: Подумайте, что должно произойти первым, а что последним.

Ответ:

- Иницилируем чтение файла с помощью `fs.promises.readFile`
- Обрабатываем ошибки через `try/catch`
- Ожидаем завершения промиса через `await`
- Работаем с данными после успешного завершения

Шаг 12:

Миссия 12

Расположите действия при одновременной загрузке нескольких файлов:

1. Создаём массив промисов для каждого файла
2. Используем `Promise.all` для ожидания всех промисов
3. Обрабатываем ошибки через `catch`
4. Обрабатываем результаты после успешного завершения всех промисов

Подсказка: Сначала формируем промисы, затем ждём их завершения и обрабатываем результат.

Ответ:

- Создаём массив промисов для каждого файла
- Используем `Promise.all` для ожидания всех промисов
- Обрабатываем ошибки через `catch`
- Обрабатываем результаты после успешного завершения всех промисов

Шаг 13:

Миссия 13

Лена хочет убедиться, что ошибки не остановят загрузку остальных файлов. Что использовать?

Подсказка: Можно обрабатывать ошибки отдельно для каждого промиса.

Ответ:

- **Добавлять `catch` для каждого промиса отдельно**
- Использовать общий `try/catch` для всех промисов
- Игнорировать ошибки
- Вызвать `setTimeout` после ошибки

Модуль 3 - Мир модулей и пакетов

Урок 1 - Работа с встроенными модулями (`fs`, `path`, `http`).

Шаг 01:

Миссия 1

Иван хочет прочитать содержимое файла асинхронно. Какой модуль Node.js ему нужен?

Подсказка: Файловая система — это именно модуль `fs`.

Ответ:

- **`fs`**
- `http`
- `path`

- **os**

Шаг 02:

Миссия 2

Мария хочет узнать абсолютный путь к файлу из относительного. Какой модуль использовать?

Подсказка: Для работы с путями файлов предназначен отдельный модуль.

Ответ:

- **path**
- fs
- http
- crypto

Шаг 03:

Миссия 3

Алексей хочет создать простой HTTP сервер. Какой модуль Node.js ему поможет?

Подсказка: Сервера создаются через встроенный модуль, который умеет работать с запросами и ответами.

Ответ:

- **http**
- fs
- path
- os

Шаг 04:

Миссия 4

Сопоставьте модули Node.js с их назначением:

Ответ:

- fs
 - path
 - http
 - os
1. Работа с файловой системой
 2. Работа с путями файлов и директорий
 3. Создание HTTP серверов
 4. Получение информации об операционной системе

Подсказка: Подумайте, какой модуль отвечает за файлы, а какой — за систему.

- **fs** — Работа с файловой системой
- **path** — Работа с путями файлов и директорий
- **http** — Создание HTTP серверов
- **os** — Получение информации об операционной системе

Шаг 05:

Миссия 5

Сопоставьте методы модуля fs с их функционалом:

Ответ:

- `fs.readFile`
- `fs.writeFile`
- `fs.readdir`
- `fs.unlink`

1. Чтение содержимого файла
2. Запись данных в файл
3. Чтение списка файлов в директории
4. Удаление файла

Подсказка: Подумайте, какие действия соответствуют чтению, записи и удалению.

- **`fs.readFile`** — Чтение содержимого файла
- **`fs.writeFile`** — Запись данных в файл
- **`fs.readdir`** — Чтение списка файлов в директории
- **`fs.unlink`** — Удаление файла

 Шаг 06:

Миссия 6

Петр хочет соединить путь к файлу из нескольких частей корректно для любой ОС. Какой метод использовать?

Подсказка: Для объединения путей есть специальная функция модуля `path`.

Ответ:

- **`path.join`**
- `fs.join`
- `http.join`
- `os.join`

 Шаг 07:

Миссия 7

Анна хочет получить расширение файла. Какой метод модуля `path` поможет?

Подсказка: Метод возвращает часть имени файла после последней точки.

Ответ:

- **`path.extname`**
- `fs.extname`
- `http.extname`
- `os.extname`

 Шаг 08:

Миссия 8

Сопоставьте методы модуля `path` с их назначением:

Ответ:

- `path.basename`
- `path.dirname`
- `path.resolve`

- `path.join`

1. Возвращает имя файла с расширением
2. Возвращает путь к директории файла
3. Возвращает абсолютный путь
4. Объединяет несколько частей пути в один

Подсказка: Подумайте, какой метод нужен для имени файла, а какой для абсолютного пути.

- **`path.basename`** — Возвращает имя файла с расширением
- **`path.dirname`** — Возвращает путь к директории файла
- **`path.resolve`** — Возвращает абсолютный путь
- **`path.join`** — Объединяет несколько частей пути в один

Шаг 09:

Миссия 9

Игорь хочет создать сервер, который возвращает текст «Hello World». Какой метод использовать для обработки запроса?

Подсказка: Метод вызывается при получении каждого запроса на сервер.

Ответ:

- **`createServer` и `callback` с `req, res`**
- `fs.readFileSync`
- `path.join`
- `console.log`

Шаг 10:

Миссия 10

Расположите шаги чтения файла через `fs.promises`:

1. Импортировать модуль `fs`
2. Вызвать `fs.promises.readFile` с путем к файлу
3. Обработать промис через `then` или `await`
4. Использовать данные после успешного завершения
5. Обработать ошибки через `catch`

Подсказка: Сначала подключаем модуль, затем читаем, потом ждём промис и обрабатываем.

Ответ:

- Импортировать модуль `fs`
- Вызвать `fs.promises.readFile` с путем к файлу
- Обработать промис через `then` или `await`
- Использовать данные после успешного завершения
- Обработать ошибки через `catch`

Шаг 11:

Миссия 11

Расположите шаги при создании HTTP сервера:

1. Импортировать модуль `http`
2. Вызвать `http.createServer` с `callback(req, res)`
3. Отправить ответ через `res.end()`

4. Запустить сервер методом `listen` на порту

Подсказка: Сначала подключаем модуль, потом создаём сервер, потом отвечаем и запускаем.

Ответ:

- Импортировать модуль `http`
- Вызвать `http.createServer` с `callback(req, res)`
- Отправить ответ через `res.end()`
- Запустить сервер методом `listen` на порту

Шаг 12:

Миссия 12

Ксения хочет проверить, существует ли файл перед чтением. Какой метод использовать?

Подсказка: Есть метод, который проверяет наличие файла и возвращает `true/false`.

Ответ:

- **`fs.existsSync`**
- `fs.readFileSync`
- `path.exists`
- `http.get`

Шаг 13:

Миссия 13

Сопоставьте свойства объекта `req` в HTTP сервере с их назначением:

Ответ:

- `req.url`
 - `req.method`
 - `req.headers`
 - `req.socket`
1. URL запроса
 2. Метод HTTP (GET, POST и т.д.)
 3. Заголовки запроса
 4. Информация о соединении

Подсказка: Подумайте, где хранится URL и метод запроса, а где сетевые данные.

- **`req.url`** — URL запроса
- **`req.method`** — Метод HTTP (GET, POST и т.д.)
- **`req.headers`** — Заголовки запроса
- **`req.socket`** — Информация о соединении

Шаг 14:

Миссия 14

Сергей хочет получить MIME тип файла при отправке через HTTP. Что использовать?

Подсказка: Для определения типа содержимого есть встроенный модуль или заголовок.

Ответ:

- **Content-Type в заголовке ответа**
- `fs.readFile`

- path.join
- req.method

Урок 2 - Внешние пакеты (lodash, dayjs и др.).

Шаг 01:

Миссия 1

Антон хочет использовать функцию для глубокого копирования объекта. Какой пакет Node.js поможет?

Подсказка: Для работы с объектами и коллекциями есть популярная библиотека.

Ответ:

- **lodash**
- dayjs
- express
- axios

Шаг 02:

Миссия 2

Мария хочет работать с датами и форматировать их красиво. Какой пакет использовать?

Подсказка: Для работы с датами и временем есть легковесный пакет.

Ответ:

- **dayjs**
- lodash
- express
- axios

Шаг 03:

Миссия 3

Сергей хочет сделать HTTP-запрос к внешнему API. Какой пакет ему поможет?

Подсказка: Популярная библиотека для работы с HTTP-запросами.

Ответ:

- **axios**
- lodash
- dayjs
- express

Шаг 04:

Миссия 4

Сопоставьте пакеты Node.js с их назначением:

Ответ:

- lodash
- dayjs
- axios

- **express**

1. Работа с объектами, массивами и функциями
2. Форматирование и работа с датами
3. Выполнение HTTP-запросов
4. Создание веб-серверов и маршрутов

Подсказка: Подумайте, какой пакет нужен для данных, а какой для сервера.

- **lodash** — Работа с объектами, массивами и функциями
- **dayjs** — Форматирование и работа с датами
- **axios** — Выполнение HTTP-запросов
- **express** — Создание веб-серверов и маршрутов

Шаг 05:

Миссия 5

Сопоставьте функции lodash с их назначением:

Ответ:

- **_cloneDeep**
 - **_merge**
 - **_debounce**
 - **_uniq**
1. Глубокое копирование объектов
 2. Слияние объектов
 3. Отложенный вызов функции
 4. Удаление дубликатов в массиве

Подсказка: Подумайте, какие функции работают с объектами, а какие с массивами.

- **_cloneDeep** — Глубокое копирование объектов
- **_merge** — Слияние объектов
- **_debounce** — Отложенный вызов функции
- **_uniq** — Удаление дубликатов в массиве

Шаг 06:

Миссия 6

Ольга хочет отслеживать время выполнения функции и выводить его в консоль. Какой пакет поможет удобно работать с датами?

Подсказка: Используется легковесная библиотека для работы с временем и датами.

Ответ:

- **dayjs**
- **lodash**
- **axios**
- **express**

Шаг 07:

Миссия 7

Владислав хочет сделать HTTP POST запрос с данными формы. Какой пакет использовать?

Подсказка: Это современная библиотека для работы с HTTP-запросами.

Ответ:

- **axios**
- **lodash**
- **dayjs**
- **express**

Шаг 08:

Миссия 8

Сопоставьте методы dayjs с их назначением:

Ответ:

- **dayjs().format()**
- **dayjs().add()**
- **dayjs().diff()**
- **dayjs().subtract()**

1. Форматирование даты в строку
2. Прибавление времени
3. Вычисление разницы между датами
4. Вычитание времени

Подсказка: Подумайте, какие методы изменяют дату, а какие возвращают строку или число.

- **dayjs().format()** — Форматирование даты в строку
- **dayjs().add()** — Прибавление времени
- **dayjs().diff()** — Вычисление разницы между датами
- **dayjs().subtract()** — Вычитание времени

Шаг 09:

Миссия 9

Ирина хочет создать массив уникальных значений из массива чисел. Какой метод lodash использовать?

Подсказка: Ищите метод, который удаляет дубликаты.

Ответ:

- **_.uniq**
- **_.merge**
- **_.cloneDeep**
- **_.debounce**

Шаг 10:

Миссия 10

Пётр хочет объединить несколько объектов в один. Какой метод lodash использовать?

Подсказка: Метод объединяет все свойства объектов в новый объект.

Ответ:

- **_.merge**
- **_.uniq**
- **_.cloneDeep**
- **_.debounce**

Шаг 11:

Миссия 11

Расположите шаги работы с HTTP-запросом через axios:

1. Импортировать axios
2. Сформировать конфигурацию запроса (URL, метод, данные)
3. Выполнить запрос axios(config) или axios.get/post
4. Обработать успешный ответ через then или await
5. Обработать ошибки через catch

Подсказка: Сначала подключаем библиотеку, потом формируем запрос, затем выполняем и обрабатываем результат.

Ответ:

- Импортировать axios
- Сформировать конфигурацию запроса (URL, метод, данные)
- Выполнить запрос axios(config) или axios.get/post
- Обработать успешный ответ через then или await
- Обработать ошибки через catch

Шаг 12:

Миссия 12

Расположите шаги форматирования даты с dayjs:

1. Импортировать dayjs
2. Создать объект даты dayjs()
3. Добавить или вычесть время при необходимости
4. Отформатировать дату методом format()

Подсказка: Сначала подключаем библиотеку, затем создаем объект, меняем дату и выводим форматированную строку.

Ответ:

- Импортировать dayjs
- Создать объект даты dayjs()
- Добавить или вычесть время при необходимости
- Отформатировать дату методом format()

Шаг 13:

Миссия 13

Николай хочет использовать функцию, чтобы ограничить частоту вызова обработчика событий. Какой lodash метод подойдет?

Подсказка: Этот метод предотвращает слишком частый вызов функции.

Ответ:

- **_.debounce**
- **_.merge**
- **_.cloneDeep**
- **_.uniq**

Урок 3 - NPM-скрипты.

Шаг 01:

Миссия 1

Антон хочет сделать глубокую копию объекта без потери вложенных данных. Какой пакет лучше использовать?

Подсказка: Для работы с объектами и массивами есть удобная библиотека.

Ответ:

- **lodash**
- dayjs
- axios
- express

Шаг 02:

Миссия 2

Мария хочет работать с датами и красиво их форматировать. Какой пакет использовать?

Подсказка: Используется легковесная библиотека для работы с временем.

Ответ:

- **dayjs**
- lodash
- axios
- express

Шаг 03:

Миссия 3

Сергей хочет сделать HTTP-запрос к внешнему API. Какой пакет выбрать?

Подсказка: Популярная библиотека для работы с HTTP-запросами.

Ответ:

- **axios**
- lodash
- dayjs
- express

Шаг 04:

Миссия 4

Сопоставьте пакеты Node.js с их назначением:

Ответ:

- lodash
- dayjs
- axios
- express

1. Работа с объектами и массивами
2. Форматирование и работа с датами
3. HTTP-запросы к API

4. Создание веб-серверов

Подсказка: Подумайте, какой пакет нужен для данных, а какой для сервера.

- **lodash** — Работа с объектами и массивами
- **dayjs** — Форматирование и работа с датами
- **axios** — HTTP-запросы к API
- **express** — Создание веб-серверов

Шаг 05:

Миссия 5

Сопоставьте функции lodash с их назначением:

Ответ:

- `_cloneDeep`
 - `_merge`
 - `_debounce`
 - `_uniq`
1. Глубокое копирование объектов
 2. Объединение объектов
 3. Отложенный вызов функции
 4. Удаление дубликатов из массива

Подсказка: Подумайте, какие функции работают с объектами, а какие с массивами.

- **`_cloneDeep`** — Глубокое копирование объектов
- **`_merge`** — Объединение объектов
- **`_debounce`** — Отложенный вызов функции
- **`_uniq`** — Удаление дубликатов из массива

Шаг 06:

Миссия 6

Ольга хочет вывести текущее время в формате 'DD-MM-YYYY'. Какой пакет использовать?

Подсказка: Подходит библиотека для работы с датами и временем.

Ответ:

- **dayjs**
- lodash
- axios
- express

Шаг 07:

Миссия 7

Сопоставьте методы dayjs с их назначением:

Ответ:

- `dayjs().format()`
- `dayjs().add()`
- `dayjs().diff()`

- `dayjs().subtract()`

1. Форматирование даты в строку
2. Прибавление времени
3. Разница между датами
4. Вычитание времени

Подсказка: Обратите внимание, какие методы изменяют дату, а какие возвращают строку или число.

- **`dayjs().format()`** — Форматирование даты в строку
- **`dayjs().add()`** — Прибавление времени
- **`dayjs().diff()`** — Разница между датами
- **`dayjs().subtract()`** — Вычитание времени

Шаг 08:

Миссия 8

Ирина хочет создать массив уникальных чисел из исходного массива. Какой метод lodash использовать?

Подсказка: Метод удаляет все дубликаты.

Ответ:

- **`_.uniq`**
- `_.merge`
- `_.cloneDeep`
- `_.debounce`

Шаг 09:

Миссия 9

Пётр хочет объединить несколько объектов в один. Какой метод lodash использовать?

Подсказка: Метод объединяет все свойства объектов.

Ответ:

- **`_.merge`**
- `_.uniq`
- `_.cloneDeep`
- `_.debounce`

Шаг 10:

Миссия 10

Расположите шаги работы с axios:

1. Импортировать axios
2. Сформировать конфигурацию запроса
3. Выполнить запрос `axios(config)` или `axios.get/post`
4. Обработать успешный ответ
5. Обработать ошибки через `catch`

Подсказка: Сначала подключаем библиотеку, затем формируем запрос, потом выполняем и обрабатываем результат.

Ответ:

- Импортировать axios
- Сформировать конфигурацию запроса
- Выполнить запрос axios(config) или axios.get/post
- Обработать успешный ответ
- Обработать ошибки через catch

 Шаг 11:

Миссия 11

Расположите шаги форматирования даты с dayjs:

1. Импортировать dayjs
2. Создать объект dayjs()
3. Добавить или вычесть время
4. Отформатировать дату методом format()

Подсказка: Сначала подключаем библиотеку, создаем объект, меняем дату и выводим строку.

Ответ:

- Импортировать dayjs
- Создать объект dayjs()
- Добавить или вычесть время
- Отформатировать дату методом format()

 Шаг 12:

Миссия 12

Николай хочет ограничить частоту вызова функции-обработчика событий. Какой lodash метод использовать?

Подсказка: Этот метод предотвращает слишком частый вызов функции.

Ответ:

- **_.debounce**
- _.merge
- _.cloneDeep
- _.uniq

 Шаг 13:

Миссия 13

Расположите шаги объединения объектов с lodash:

1. Импортировать lodash
2. Создать объекты для объединения
3. Вызвать _.merge(obj1, obj2)
4. Использовать объединённый объект в коде

Подсказка: Сначала библиотека, затем объекты, потом объединяем и применяем.

Ответ:

- Импортировать lodash
- Создать объекты для объединения
- Вызвать _.merge(obj1, obj2)

- Использовать объединённый объект в коде

🔧 Шаг 14:

Миссия 14

Расположите шаги для ограничения вызова функции с `_debounce`:

1. Импортировать `lodash`
2. Создать функцию-обработчик
3. Применить `_debounce` к функции
4. Привязать обработчик к событию

Подсказка: Сначала библиотека, затем функция, потом ограничение частоты, затем событие.

Ответ:

- Импортировать `lodash`
- Создать функцию-обработчик
- Применить `_debounce` к функции
- Привязать обработчик к событию

📖 Урок 4 - Сборка конфигурации `dotenv`, `config`.

🔧 Шаг 01:

Миссия 1

Алексей хочет хранить секреты приложения (API-ключи, пароли) вне кода. Какой пакет лучше использовать?

Подсказка: Идеально подходит для хранения переменных окружения.

Ответ:

- **`dotenv`**
- `config`
- `lodash`
- `axios`

🔧 Шаг 02:

Миссия 2

Марина хочет структурировать конфигурацию приложения для разных сред (`dev`, `prod`). Какой пакет выбрать?

Подсказка: Позволяет иметь отдельные конфиги для разных сред.

Ответ:

- **`config`**
- `dotenv`
- `express`
- `axios`

🔧 Шаг 03:

Миссия 3

Игорь хочет загрузить переменные из `.env` в `process.env` автоматически при старте приложения. Какой метод `dotenv` использовать?

Подсказка: Обычно это делается в начале приложения.

Ответ:

- `require('dotenv').config()`
- `dotenv.load()`
- `config.load()`
- `require('dotenv').loadEnv()`

Шаг 04:

Миссия 4

Сопоставьте пакет с его основным назначением:

Ответ:

- `dotenv`
 - `config`
1. Загрузка переменных окружения из файла `.env`
 2. Управление конфигурацией для разных сред (`dev`, `prod`, `test`)

Подсказка: Подумайте, что хранится в `.env`, а что — в отдельных конфигурационных файлах.

- **`dotenv`** — Загрузка переменных окружения из файла `.env`
- **`config`** — Управление конфигурацией для разных сред (`dev`, `prod`, `test`)

Шаг 05:

Миссия 5

Сопоставьте переменные окружения с типичными данными:

Ответ:

- `DB_HOST`
 - `DB_PASSWORD`
 - `API_KEY`
 - `PORT`
1. Адрес базы данных
 2. Пароль к базе данных
 3. Секретный ключ для доступа к API
 4. Порт, на котором слушает сервер

Подсказка: Названия часто отражают содержимое переменной.

- **`DB_HOST`** — Адрес базы данных
- **`DB_PASSWORD`** — Пароль к базе данных
- **`API_KEY`** — Секретный ключ для доступа к API
- **`PORT`** — Порт, на котором слушает сервер

Шаг 06:

Миссия 6

Аня хочет, чтобы приложение автоматически подхватывало переменные окружения без ручного импорта каждого значения. Как это сделать?

Подсказка: Используется пакет, который читает `.env` при старте.

Ответ:

- **Использовать `require('dotenv').config()` в начале приложения**

- Использовать `config.get()` для каждой переменной
- Прописать переменные вручную в коде
- Создать отдельный модуль с переменными

🔧 Шаг 07:

Миссия 7

Сергей хочет использовать разные конфигурации для `development` и `production`. Как организовать файлы с `config`?

Подсказка: `config` поддерживает структуру папок для сред.

Ответ:

- Создать папку `config` с файлами `default.json`, `development.json`, `production.json`
- Создать один файл `.env` для всех сред
- Хранить конфиги в БД
- Прописывать все значения в коде вручную

🔧 Шаг 08:

Миссия 8

Сопоставьте `config`-методы с их назначением:

Ответ:

- `config.get()`
 - `config.has()`
 - `config.util.loadFileConfigs()`
1. Получение значения по ключу
 2. Проверка существования ключа
 3. Загрузка конфигурационных файлов вручную

Подсказка: Подумайте, какие методы читают данные, а какие проверяют наличие.

- **`config.get()`** — Получение значения по ключу
- **`config.has()`** — Проверка существования ключа
- **`config.util.loadFileConfigs()`** — Загрузка конфигурационных файлов вручную

🔧 Шаг 09:

Миссия 9

Влад хочет защитить приложение от отсутствия необходимых переменных окружения. Как это сделать?

Подсказка: Сначала нужно проверить наличие всех обязательных переменных.

Ответ:

- Использовать `config.has()` или проверку `process.env`
- Игнорировать проверку
- Писать все значения вручную
- Создать отдельный файл с дефолтами без проверки

🔧 Шаг 10:

Миссия 10

Елена хочет хранить секретные ключи и одновременно иметь разные настройки для dev и prod. Как лучше организовать?

Подсказка: Можно объединить .env и config.

Ответ:

- **Секреты в .env, остальные настройки в config**
- Всё хранить в .env
- Всё хранить в config
- Хранить все переменные в коде

Шаг 11:

Миссия 11

Расположите шаги использования dotenv в правильном порядке:

1. Установить пакет dotenv
2. Создать файл .env и прописать переменные
3. Подключить dotenv в начале приложения с require('dotenv').config()
4. Использовать переменные через process.env

Подсказка: Сначала установка, потом создание, подключение и использование.

Ответ:

- Установить пакет dotenv
- Создать файл .env и прописать переменные
- Подключить dotenv в начале приложения с require('dotenv').config()
- Использовать переменные через process.env

Урок 5 - Мини-босс написать REST API без фреймворков (чистый http).

Шаг 01:

Миссия 1

Алекс хочет создать простой сервер на Node.js без фреймворков. Какой модуль использовать?

Подсказка: Это встроенный модуль для HTTP.

Ответ:

- **http**
- express
- fs
- path

Шаг 02:

Миссия 2

Маша хочет, чтобы сервер слушал порт 3000. Какой метод использовать?

Подсказка: Сначала создаем сервер, потом запускаем его на порту.

Ответ:

- **server.listen(3000)**
- http.createServer(3000)
- server.start(3000)
- server.run(3000)

🔧 Шаг 03:

Миссия 3

Сергей хочет обрабатывать GET-запросы на путь /users. Что проверять в callback сервере?

Подсказка: Обратите внимание на метод и url объекта запроса.

Ответ:

- **req.method === 'GET' && req.url === '/users'**
- **res.method === 'GET'**
- **req.type === 'GET' && req.path === '/users'**
- **server.url === '/users'**

🔧 Шаг 04:

Миссия 4

Сопоставьте объекты и их назначение в чистом Node.js сервере:

Ответ:

- req
 - res
 - server
1. Объект запроса, содержит метод и URL
 2. Объект ответа, через который отправляем данные клиенту
 3. HTTP-сервер, который слушает порт

Подсказка: Подумайте, что приходит от клиента, что от сервера, а что управляет всем приложением.

- **req** — Объект запроса, содержит метод и URL
- **res** — Объект ответа, через который отправляем данные клиенту
- **server** — HTTP-сервер, который слушает порт

🔧 Шаг 05:

Миссия 5

Сопоставьте методы объекта res с их назначением:

Ответ:

- res.writeHead()
 - res.end()
 - res.write()
1. Установка заголовков и статус-кода ответа
 2. Закрытие ответа и отправка данных клиенту
 3. Отправка части тела ответа

Подсказка: Сначала настраиваем заголовки, потом отправляем данные и закрываем ответ.

- **res.writeHead()** — Установка заголовков и статус-кода ответа
- **res.end()** — Закрытие ответа и отправка данных клиенту
- **res.write()** — Отправка части тела ответа

🔧 Шаг 06:

Миссия 6

Аня хочет возвращать JSON с пользователями. Как правильно установить заголовок?

Подсказка: Обратите внимание на Content-Type.

Ответ:

- **res.writeHead(200, {'Content-Type': 'application/json'})**
- res.writeHead(200, {'Content-Type': 'text/html'})
- res.setHeader('Content-Type', 'application/json')
- res.header('application/json')

🔧 Шаг 07:

Миссия 7

Игорь хочет обрабатывать POST-запросы с телом. Как получить данные из req?

Подсказка: Данные приходят по частям и нужно собрать их в буфер или строку.

Ответ:

- **Слушать событие 'data' и 'end' на req**
- req.body сразу содержит данные
- req.read()
- Использовать res.data()

🔧 Шаг 08:

Миссия 8

Сопоставьте событие с его назначением на объекте req:

Ответ:

- 'data'
 - 'end'
1. Приходит часть тела запроса
 2. Завершение передачи данных

Подсказка: Сначала данные приходят по частям, потом сигнал о завершении.

- **'data'** — Приходит часть тела запроса
- **'end'** — Завершение передачи данных

🔧 Шаг 09:

Миссия 9

Аня хочет возвращать JSON с пользователями. Как правильно установить заголовок?

Подсказка: Обратите внимание на Content-Type.

Ответ:

- **res.writeHead(200, {'Content-Type': 'application/json'})**
- res.writeHead(200, {'Content-Type': 'text/html'})
- res.setHeader('Content-Type', 'application/json')
- res.header('application/json')

🔮 Шаг 10:

Миссия 10

Игорь хочет обрабатывать POST-запросы с телом. Как получить данные из req?

Подсказка: Данные приходят по частям и нужно собрать их в буфер или строку.

Ответ:

- **Слушать событие 'data' и 'end' на req**
- req.body сразу содержит данные
- req.read()
- Использовать res.data()

🔮 Шаг 11:

Миссия 11

Сопоставьте событие с его назначением на объекте req:

Ответ:

- 'data'
 - 'end'
1. Приходит часть тела запроса
 2. Завершение передачи данных

Подсказка: Сначала данные приходят по частям, потом сигнал о завершении.

- **'data'** — Приходит часть тела запроса
- **'end'** — Завершение передачи данных

🔮 Шаг 12:

Миссия 12

Вика хочет обрабатывать ошибки сервера и возвращать 500. Как это сделать?

Подсказка: Используйте статус-код и короткое сообщение.

Ответ:

- **res.writeHead(500, {'Content-Type': 'text/plain'}); res.end('Internal Server Error')**
- res.status(500).send('Error')
- res.end(500)
- res.write(500)

🔮 Шаг 13:

Миссия 13

Расположите действия при обработке POST-запроса в правильном порядке:

1. Создать сервер
2. Слушать событие 'data' и собирать части тела запроса
3. Слушать событие 'end' и обработать полный запрос
4. Отправить ответ клиенту

Подсказка: Сначала создаем сервер, потом собираем данные, после обработки — отправляем ответ.

Ответ:

- Создать сервер
- Слушать событие 'data' и собирать части тела запроса
- Слушать событие 'end' и обработать полный запрос
- Отправить ответ клиенту

Модуль 4 - Башня авторизации

Урок 1 - JWT, refresh-токены.

Шаг 01:

Миссия 1

Вы только что создали API с Node.js. Какой механизм аутентификации безопаснее всего использовать для веб-приложений?

Подсказка: Рассмотрите вариант, который позволяет серверу проверять идентичность без хранения сессий.

Ответ:

- **JWT (JSON Web Token)**
- Сессионные куки без токена
- Basic Auth с паролем в URL
- OAuth только с client secret на фронтенде

Шаг 02:

Миссия 2

Когда мы создаем JWT для пользователя, что обычно включается в payload?

Подсказка: Подумайте о данных, которые сервер должен использовать для проверки прав пользователя.

Ответ:

- **Идентификатор пользователя и срок действия токена**
- Пароль пользователя
- Секретный ключ сервера
- IP адрес клиента

Шаг 03:

Миссия 3

Вы настроили JWT с коротким временем жизни. Как правильно обновлять токен после его истечения?

Подсказка: Подумайте о механизме, который позволяет не вводить логин повторно.

Ответ:

- **Использовать refresh-токен для получения нового JWT**
- Отправить повторный логин пользователя на фронтенде
- Игнорировать истечение токена и продолжать работу
- Отправлять JWT повторно через cookie без проверки

🔧 Шаг 04:

Миссия 4

📖 Сопоставьте термин JWT с его определением.

Ответ:

- Header
 - Payload
 - Signature
 - Refresh Token
1. Содержит алгоритм шифрования и тип токена.
 2. Содержит данные пользователя и срок действия токена.
 3. Гарантирует целостность токена и предотвращает подделку.
 4. Используется для получения нового JWT без повторной аутентификации.

💡 Подсказка:

Каждая часть токена имеет свою роль: заголовок для алгоритма, полезная нагрузка для данных, подпись для безопасности, а refresh-токен для обновления.

- **Header** — Содержит алгоритм шифрования и тип токена.
- **Payload** — Содержит данные пользователя и срок действия токена.
- **Signature** — Гарантирует целостность токена и предотвращает подделку.
- **Refresh Token** — Используется для получения нового JWT без повторной аутентификации.

🔧 Шаг 05:

Миссия 5

📖 Сопоставьте метод Express.js с его назначением для аутентификации JWT.

Ответ:

- `jsonwebtoken.sign()`
 - `jsonwebtoken.verify()`
 - Middleware проверки токена
 - Endpoint для refresh-токена
1. Создает новый JWT с данными пользователя.
 2. Проверяет валидность токена.
 3. Проверяет токен на каждом запросе к защищенному ресурсу.
 4. Обрабатывает запрос на получение нового JWT с использованием refresh-токена.

💡 Подсказка:

Подумайте, кто создаёт, кто проверяет, и как сервер обновляет токены.

- **`jsonwebtoken.sign()`** — Создает новый JWT с данными пользователя.
- **`jsonwebtoken.verify()`** — Проверяет валидность токена.
- **Middleware проверки токена** — Проверяет токен на каждом запросе к защищенному ресурсу.
- **Endpoint для refresh-токена** — Обрабатывает запрос на получение нового JWT с использованием refresh-токена.

🔧 Шаг 06:

Миссия 6

Какая из следующих практик повышает безопасность JWT?

Подсказка: Подумайте о том, что делает токен менее уязвимым к подделке.

Ответ:

- **Использовать короткий срок жизни и refresh-токены**
- Хранить секретный ключ в коде фронтенда
- Не проверять подпись токена
- Отправлять токен через GET параметры URL

Шаг 07:

Миссия 7

Что произойдет, если refresh-токен скомпрометирован?

Подсказка: Подумайте о том, что злоумышленник сможет делать с токеном, если он действителен.

Ответ:

- **Злоумышленник сможет получать новые JWT от имени пользователя**
- JWT перестанут работать
- Refresh-токен автоматически заблокируется
- Пользователь автоматически выйдет из системы

Шаг 08:

Миссия 8

 Сопоставьте тип атаки с описанием при использовании JWT.

Ответ:

- Replay Attack
 - Token Forgery
 - Token Theft
 - Brute Force Signature
1. Использование украденного токена для доступа к ресурсам.
 2. Попытка угадать или подделать подпись токена.
 3. Повторная отправка ранее перехваченного запроса с токеном.
 4. Создание поддельного токена с целью обмана сервера.

 **Подсказка:**

Обратите внимание, что атаки могут быть связаны с перехватом, подделкой или угадыванием токена.

- **Replay Attack** — Повторная отправка ранее перехваченного запроса с токеном.
- **Token Forgery** — Создание поддельного токена с целью обмана сервера.
- **Token Theft** — Использование украденного токена для доступа к ресурсам.
- **Brute Force Signature** — Попытка угадать или подделать подпись токена.

Шаг 09:

Миссия 9

Вы хотите ограничить доступ к определённым API только для админов. Как это лучше реализовать с JWT?

Подсказка: Подумайте о хранении роли пользователя внутри токена.

Ответ:

- **Добавить роль пользователя в payload JWT и проверять её на сервере**
- Проверять роль только в базе данных без токена
- Отправлять роль пользователя через URL параметры
- Хранить роль в localStorage и доверять фронтенду

Шаг 10:

Миссия 10

 Сопоставьте заголовки HTTP с их использованием при работе с JWT.

Ответ:

- Authorization
 - Set-Cookie
 - Content-Type
 - X-Refresh-Token
1. Передача JWT при каждом запросе
 2. Отправка refresh-токена от сервера к клиенту
 3. Указывает тип передаваемых данных
 4. Дополнительный заголовок для refresh-токена в некоторых схемах

Подсказка:

JWT могут передаваться через разные заголовки или куки, важно понимать их назначение.

- **Authorization** — Передача JWT при каждом запросе
- **Set-Cookie** — Отправка refresh-токена от сервера к клиенту
- **Content-Type** — Указывает тип передаваемых данных
- **X-Refresh-Token** — Дополнительный заголовок для refresh-токена в некоторых схемах

Шаг 11:

Миссия 11

Как можно защитить refresh-токены на клиентской стороне?

Подсказка: Думайте о безопасном хранении токена, чтобы злоумышленник не получил доступ.

Ответ:

- **Хранить в HttpOnly cookie**
- Хранить в localStorage
- Хранить в sessionStorage
- Передавать через URL каждый раз

Шаг 12:

Миссия 12

Ваш коллега спрашивает: "Можно ли использовать один JWT на всех устройствах одновременно?" Как правильно ответить?

Подсказка: Подумайте о безопасности и управлении сессиями.

Ответ:

- **Лучше создавать отдельные JWT для каждого устройства**
- Да, один JWT подойдет для всех устройств
- Можно использовать один токен, если он короткий

- Токены не нужны, если есть пароль

Урок 2 - Cookies и сессии.

Шаг 01:

Миссия 1

Вы создаете веб-приложение на Node.js. Какой способ хранения информации о пользователе после входа в систему наиболее распространен?

Подсказка: Подумайте о способе, который сервер и браузер могут использовать совместно для идентификации.

Ответ:

- **Cookies**
- LocalStorage
- Сессии без cookie
- Query-параметры URL

Шаг 02:

Миссия 2

Вы настроили сессии с помощью express-session. Какой параметр помогает защитить cookie от доступа JavaScript на фронтенде?

Подсказка: Подумайте о свойствах cookie, которые ограничивают доступ к ним из скриптов.

Ответ:

- **HttpOnly**
- Secure
- SameSite
- Path

Шаг 03:

Миссия 3

Вы хотите, чтобы cookie передавались только через HTTPS. Какое свойство нужно включить?

Подсказка: Думайте о безопасности передачи данных.

Ответ:

- **Secure**
- HttpOnly
- Max-Age
- Domain

Шаг 04:

Миссия 4

 Сопоставьте понятие с его описанием.

Ответ:

- Session
- Cookie
- HttpOnly

- SameSite

1. Хранит данные на сервере о состоянии пользователя между запросами.
2. Маленький фрагмент данных, сохраняемый браузером на клиенте.
3. Свойство cookie, запрещающее доступ через JavaScript.
4. Свойство cookie, ограничивающее отправку данных между сайтами.

Подсказка:

Обратите внимание на различие между серверным и клиентским хранением данных и на безопасность.

- **Session** — Хранит данные на сервере о состоянии пользователя между запросами.
- **Cookie** — Маленький фрагмент данных, сохраняемый браузером на клиенте.
- **HttpOnly** — Свойство cookie, запрещающее доступ через JavaScript.
- **SameSite** — Свойство cookie, ограничивающее отправку данных между сайтами.

Шаг 05:

Миссия 5

 Сопоставьте тип cookie с его назначением.

Ответ:

- Session Cookie
 - Persistent Cookie
 - Secure Cookie
 - HttpOnly Cookie
1. Сохраняется только на время сессии и удаляется после закрытия браузера.
 2. Сохраняется на клиенте после закрытия браузера, пока не истечет срок действия.
 3. Передается только через HTTPS соединение.
 4. Не доступен через JavaScript на фронтенде.

Подсказка:

Сфокусируйтесь на сроке жизни и безопасности cookie.

- **Session Cookie** — Сохраняется только на время сессии и удаляется после закрытия браузера.
- **Persistent Cookie** — Сохраняется на клиенте после закрытия браузера, пока не истечет срок действия.
- **Secure Cookie** — Передается только через HTTPS соединение.
- **HttpOnly Cookie** — Не доступен через JavaScript на фронтенде.

Шаг 06:

Миссия 6

Вы заметили, что пользователи теряют сессии после закрытия браузера. Как это исправить?

Подсказка: Подумайте о типе cookie, который переживает закрытие браузера.

Ответ:

- **Использовать persistent cookie с заданным Max-Age**
- Удалить HttpOnly свойство
- Удалить SameSite
- Сохранять сессию только в памяти сервера

Шаг 07:

Миссия 7

Как предотвратить подделку cookie (CSRF) при работе с сессиями?

Подсказка: Подумайте о способах ограничения отправки cookie между сайтами.

Ответ:

- Установить **SameSite=strict** или **Lax**
- Хранить сессию только на сервере
- Отключить **Secure**
- Использовать обычные сессионные cookie без свойств

Шаг 08:

Миссия 8

 Сопоставьте атрибут cookie с его эффектом.

Ответ:

- Domain
 - Path
 - Max-Age
 - Expires
1. Определяет, на каких поддоменах доступен cookie.
 2. Указывает путь, при котором cookie отправляется на сервер.
 3. Срок жизни cookie в секундах с момента создания.
 4. Дата, когда cookie станет недействительным.

 **Подсказка:**

Сосредоточьтесь на сроке действия и области действия cookie.

- **Domain** — Определяет, на каких поддоменах доступен cookie.
- **Path** — Указывает путь, при котором cookie отправляется на сервер.
- **Max-Age** — Срок жизни cookie в секундах с момента создания.
- **Expires** — Дата, когда cookie станет недействительным.

Шаг 09:

Миссия 9

 Сопоставьте термин с примером из жизни программиста.

Ответ:

- Session Hijacking
 - Cookie Theft
 - Secure Cookie
 - HttpOnly Cookie
1. Злоумышленник получает доступ к сессии пользователя, подменяя идентификатор.
 2. Кража cookie через XSS-атаку на сайте.
 3. Cookie передается только по HTTPS.
 4. Cookie недоступен для скриптов, защищая от XSS.

 **Подсказка:**

Обратите внимание на угрозы и методы защиты cookie.

- **Session Hijacking** — Злоумышленник получает доступ к сессии пользователя, подменяя идентификатор.
- **Cookie Theft** — Кража cookie через XSS-атаку на сайте.
- **Secure Cookie** — Cookie передается только по HTTPS.
- **HttpOnly Cookie** — Cookie недоступен для скриптов, защищая от XSS.

🔮 Шаг 10:

Миссия 10

Ваш коллега спрашивает: "Можно ли использовать одну сессию на нескольких устройствах одновременно?" Как лучше ответить?

Подсказка: Подумайте о безопасности и возможности управления сессиями.

Ответ:

- **Лучше создавать отдельные сессии для каждого устройства**
- Одна сессия подходит для всех устройств
- Можно использовать одну сессию, если включен HttpOnly
- Сессии не нужны, если есть cookies

🔮 Шаг 11:

Миссия 11

Что происходит с сессией на сервере, если пользователь закрывает браузер, а cookie session только временные?

Подсказка: Подумайте, что происходит с временными данными, привязанными к браузеру.

Ответ:

- **Сессия исчезает после закрытия браузера**
- Сессия остается на сервере и доступна при новом запуске
- Сессия автоматически становится persistent
- Сессия обновляется сама

🔮 Шаг 12:

Миссия 12

Вы хотите защитить cookie от межсайтовых скриптов. Какое свойство нужно использовать?

Подсказка: Думайте о защите от XSS и CSRF.

Ответ:

- **HttpOnly**
- Max-Age
- Secure
- Domain

🔮 Шаг 13:

Миссия 13

Расположите шаги работы сессий в правильном порядке:

1. Пользователь отправляет логин и пароль
2. Сервер создает сессию и присваивает идентификатор
3. Идентификатор сессии сохраняется в cookie и отправляется клиенту

4. При последующих запросах сервер проверяет идентификатор cookie

Подсказка:

Сначала пользователь идентифицируется, затем сервер создаёт сессию, cookie отправляется клиенту, и сервер проверяет её при каждом запросе.

Ответ:

- Пользователь отправляет логин и пароль
- Сервер создает сессию и присваивает идентификатор
- Идентификатор сессии сохраняется в cookie и отправляется клиенту
- При последующих запросах сервер проверяет идентификатор cookie

Урок 3 - RBAC и роли пользователей.

Шаг 01:

Миссия 1

Вы внедряете RBAC в новое приложение. Что означает роль "admin" в классической схеме?

Подсказка: Подумайте о полном доступе к функционалу.

Ответ:

- **Полный доступ к системе и управлению пользователями**
- Просмотр ограниченной информации
- Только создание контента
- Нет доступа ни к чему

Шаг 02:

Миссия 2

Какую роль обычно назначают обычным пользователям без административных прав?

Подсказка: Подумайте о минимально необходимом доступе для работы.

Ответ:

- **User**
- Admin
- Superuser
- Guest

Шаг 03:

Миссия 3

Вы добавляете роль "moderator" для контроля комментариев. Какой тип прав логичен для такой роли?

Подсказка: Подумайте о том, какие действия нужны для контроля контента.

Ответ:

- **Редактирование и удаление пользовательского контента**
- Создание новых ролей
- Доступ к базе данных пользователей
- Полный доступ к настройкам сервера

Шаг 04:

Миссия 4

 Сопоставьте роль с её назначением в системе.

Ответ:

- Admin
- User
- Moderator
- Guest

1. Полный контроль над системой, включая управление пользователями.
2. Ограниченный доступ для выполнения основных задач.
3. Модерация и контроль пользовательского контента.
4. Минимальный доступ, просмотр публичной информации.

 **Подсказка:**

Подумайте, кто отвечает за настройку системы, кто за обычные операции и кто за модерацию.

- **Admin** — Полный контроль над системой, включая управление пользователями.
- **User** — Ограниченный доступ для выполнения основных задач.
- **Moderator** — Модерация и контроль пользовательского контента.
- **Guest** — Минимальный доступ, просмотр публичной информации.

 **Шаг 05:**

Миссия 5

 Сопоставьте принцип RBAC с его объяснением.

Ответ:

- Принцип наименьших привилегий
 - Назначение ролей
 - Иерархия ролей
 - Разграничение доступа по действиям
1. Пользователь получает только те права, которые необходимы для работы.
 2. Группировка пользователей по ролям для удобного управления.
 3. Роли могут наследовать права друг от друга.
 4. Контроль прав на уровне конкретных действий в системе.

 **Подсказка:**

Подумайте о безопасности, удобстве управления и иерархии.

- **Принцип наименьших привилегий** — Пользователь получает только те права, которые необходимы для работы.
- **Назначение ролей** — Группировка пользователей по ролям для удобного управления.
- **Иерархия ролей** — Роли могут наследовать права друг от друга.
- **Разграничение доступа по действиям** — Контроль прав на уровне конкретных действий в системе.

 **Шаг 06:**

Миссия 6

Вы хотите, чтобы менеджеры имели доступ к отчетам, но не могли изменять настройки пользователей. Какую роль им назначить?

Подсказка: Подумайте о разделении доступа между управлением контентом и администрированием.

Ответ:

- **Manager**
- Admin
- User
- Guest

 **Шаг 07:**

Миссия 7

Какой подход помогает быстро добавлять новые роли в RBAC систему?

Подсказка: Подумайте о шаблонах и группах прав.

Ответ:

- **Создание шаблонов ролей с набором прав**
- Назначение прав каждому пользователю вручную
- Использование одного Admin для всех задач
- Игнорирование прав и доверие пользователям

 **Шаг 08:**

Миссия 8

 Сопоставьте действия с ролями, которым они логично доступны.

Ответ:

- Изменение паролей пользователей
- Просмотр публичного контента
- Удаление комментариев
- Доступ к финансовым отчетам

1. Admin
2. Guest
3. Moderator
4. Manager

 **Подсказка:**

Обратите внимание, кто отвечает за безопасность, кто за модерацию и кто за данные.

- **Изменение паролей пользователей** — Admin
- **Просмотр публичного контента** — Guest
- **Удаление комментариев** — Moderator
- **Доступ к финансовым отчетам** — Manager

 **Шаг 09:**

Миссия 9

 Сопоставьте тип права с примером его использования.

Ответ:

- Read
- Write
- Delete

- Execute

1. Просмотр списка пользователей
2. Редактирование профиля пользователя
3. Удаление комментария на сайте
4. Запуск фонового процесса или скрипта

Подсказка:

Думайте о CRUD действиях и правах на выполнение кода.

- **Read** — Просмотр списка пользователей
- **Write** — Редактирование профиля пользователя
- **Delete** — Удаление комментария на сайте
- **Execute** — Запуск фонового процесса или скрипта

Шаг 10:

Миссия 10

Какой принцип нужно соблюдать, чтобы минимизировать риск утечки данных через RBAC?

Подсказка: Подумайте о минимально необходимом доступе.

Ответ:

- **Принцип наименьших привилегий**
- Все пользователи получают полный доступ
- Одинаковые права для всех ролей
- Использовать только Guest роль

Урок 4 - Мини-босс реализовать регистрацию + роли (админюзер).

Шаг 01:

Миссия 1

Вы создаете первую версию регистрации. Что обязательно нужно хранить в базе для каждого пользователя?

Подсказка: Подумайте о том, чтобы пользователь мог потом войти.

Ответ:

- **Email и хэш пароля**
- Пароль в открытом виде
- Имя пользователя и адрес
- Только роль пользователя

Шаг 02:

Миссия 2

Что нужно сделать после того, как пользователь вводит пароль при регистрации?

Подсказка: Никогда не храните пароль в открытом виде.

Ответ:

- **Применить хэширование пароля**
- Сохранить его напрямую в базе
- Отправить пользователю на email
- Записать в файл на сервере

Шаг 03:

Миссия 3

При логине пользователь вводит пароль. Как система проверяет правильность?

Подсказка: Сравниваются не открытые пароли, а...

Ответ:

- **Сравнивается хэш введенного пароля с хэшем в базе**
- Сравнивается введенный пароль напрямую
- Отправляется email для подтверждения
- Используется только имя пользователя

Шаг 04:

Миссия 4

 Сопоставьте элементы системы аутентификации с их назначением.

Ответ:

- Email
 - Пароль
 - Роль
 - Хэш пароля
1. Идентификация пользователя
 2. Определение уровня доступа
 3. Секретная строка для проверки личности
 4. Безопасное хранение пароля

 **Подсказка:**

Подумайте, что отвечает за идентификацию, а что за доступ.

- **Email** — Идентификация пользователя
- **Пароль** — Секретная строка для проверки личности
- **Роль** — Определение уровня доступа
- **Хэш пароля** — Безопасное хранение пароля

Шаг 05:

Миссия 5

 Сопоставьте типы пользователей с типичными действиями.

Ответ:

- Admin
 - User
 - Guest
1. Полный доступ к настройкам и управлению пользователями
 2. Создание контента и работа с собственными данными
 3. Просмотр общедоступной информации

 **Подсказка:**

Подумайте, кто может управлять системой, а кто только работать с контентом.

- **Admin** — Полный доступ к настройкам и управлению пользователями

- **User** — Создание контента и работа с собственными данными
- **Guest** — Просмотр общедоступной информации

Шаг 06:

Миссия 6

Какой подход безопаснее для хранения пароля: соль + хэш или только хэш?

Подсказка: Соль предотвращает атаки с предвычисленными таблицами.

Ответ:

- **Соль + хэш**
- Только хэш
- Пароль в открытом виде
- MD5 без соли

Шаг 07:

Миссия 7

Что нужно сделать, чтобы пользователь с ролью Admin мог управлять другими пользователями?

Подсказка: Подумайте о доступе к таблице пользователей и правах изменения.

Ответ:

- **Назначить роль Admin с правами CRUD на пользователей**
- Назначить роль User
- Удалить таблицу пользователей
- Дать доступ Guest

Шаг 08:

Миссия 8

 Сопоставьте сценарий с правильной реакцией системы при логине.

Ответ:

- Правильный email и пароль
 - Неверный пароль
 - Незарегистрированный email
1. Разрешить доступ и создать сессию
 2. Выдать сообщение о неправильном пароле
 3. Выдать сообщение о необходимости регистрации

 **Подсказка:**

Подумайте о стандартной логике аутентификации.

- **Правильный email и пароль** — Разрешить доступ и создать сессию
- **Неверный пароль** — Выдать сообщение о неправильном пароле
- **Незарегистрированный email** — Выдать сообщение о необходимости регистрации

Шаг 09:

Миссия 9

Почему важно проверять роль пользователя перед доступом к административной странице?

Подсказка: Без проверки любой пользователь может получить полный доступ.

Ответ:

- **Для ограничения доступа и безопасности системы**
- Чтобы пользователь видел красивый интерфейс
- Для уменьшения нагрузки на сервер
- Чтобы сэкономить место в базе

🔗 Шаг 10:

Миссия 10

Расположите шаги регистрации пользователя в правильном порядке:

1. Ввод email и пароля
2. Проверка уникальности email
3. Хэширование пароля
4. Сохранение данных в базе
5. Присвоение роли

🔗 Подсказка:

Подумайте, какие действия должны идти перед сохранением, а какие после.

Ответ:

- Ввод email и пароля
- Проверка уникальности email
- Хэширование пароля
- Сохранение данных в базе
- Присвоение роли

🔗 Шаг 11:

Миссия 11

Расположите шаги логина пользователя:

1. Ввод email и пароля
2. Проверка существования email в базе
3. Сравнение хэша пароля
4. Создание сессии при успешной аутентификации
5. Вывод ошибки при неверных данных

🔗 Подсказка:

Подумайте о логике проверки данных и создании сессии.

Ответ:

- Ввод email и пароля
- Проверка существования email в базе
- Сравнение хэша пароля
- Создание сессии при успешной аутентификации
- Вывод ошибки при неверных данных

🔗 Шаг 12:

Миссия 12

Расположите действия при назначении ролей:

1. Создание списка ролей
2. Выбор пользователя
3. Присвоение выбранной роли
4. Сохранение изменений в базе

 **Подсказка:**

Подумайте о последовательности: сначала есть роли, потом выбираем пользователя, потом сохраняем.

Ответ:

- Создание списка ролей
- Выбор пользователя
- Присвоение выбранной роли
- Сохранение изменений в базе